University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO1
Lab Title: Input/Output, Data Types, Operators, Conditional Statements (If/Else)	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric

Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 1

Lab Goals / Objectives:

• Understand the basic structure of a Python program

• Learn to use the print() function for output with formatting options

• Understand Python data types: int, float, bool, string

• Learn to use the input() function for user input

• Apply sep and end parameters in print() for custom formatting

• Use type() function to identify variable data types

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. The print() Function

The print() function in Python is used to output data to the standard output device (screen). It is one of the most frequently used built-in functions in Python. The basic syntax is:

```
print(value1, value2, ..., sep=' ', end='\n')
```

The print() function converts the arguments to strings, writes them to the console, and adds a newline at the end by default. It can take multiple arguments separated by commas.

2. The sep Parameter

The sep parameter in print() specifies how to separate multiple values. By default, it is a space character ' '. You can customize it to any string:

```
print('UET', 'Lahore', sep='-')  # Output: UET-Lahore
```

3. The end Parameter

The end parameter specifies what to print at the end of the output. By default, it is a newline character '\n'. Changing it allows continuing output on the same line:

```
print('Hello', end=' ')
print('World')  # Output: Hello World
```

4. Escape Sequences

Escape sequences are special characters that begin with a backslash (\) and have special meaning in strings:

• \n - Newline: Moves the cursor to the next line

• \t - Tab: Inserts a horizontal tab

• \\ - Backslash: Prints a literal backslash

• \' - Single quote: Prints a single quote

• \" - Double quote: Prints a double quote

5. The input() Function

The input() function allows user input from the keyboard. It always returns a string value, which must be converted to the desired data type using type casting functions:

```
name = input('Enter your name: ')
age = int(input('Enter your age: '))
```

6. Python Data Types

Python has several built-in data types:

• int (Integer): Whole numbers without decimal points. Example: x = 10

• float (Floating-point): Numbers with decimal points. Example: y = 3.14

• bool (Boolean): Logical values True or False. Example: z = True

• str (String): Sequence of characters enclosed in quotes. Example: name = 'UET'

7. Type Casting

Type casting is the process of converting one data type to another:

• int() - Converts a value to an integer

• float() - Converts a value to a floating-point number

• str() - Converts a value to a string

• bool() - Converts a value to a boolean

8. The type() Function

The type() function returns the data type of a variable or value:

```
x = 10
print(type(x))  # Output: <class 'int'>
```

9. String Formatting with f-strings

f-strings (formatted string literals) provide a concise way to embed expressions inside string literals using curly braces:

```
name = 'Hashir'
print(f'Name: {name}')
```

Lab Work:

Task 1: Print Your Information

Write a Python program that prints your personal information including name, department, and university using the print() function.

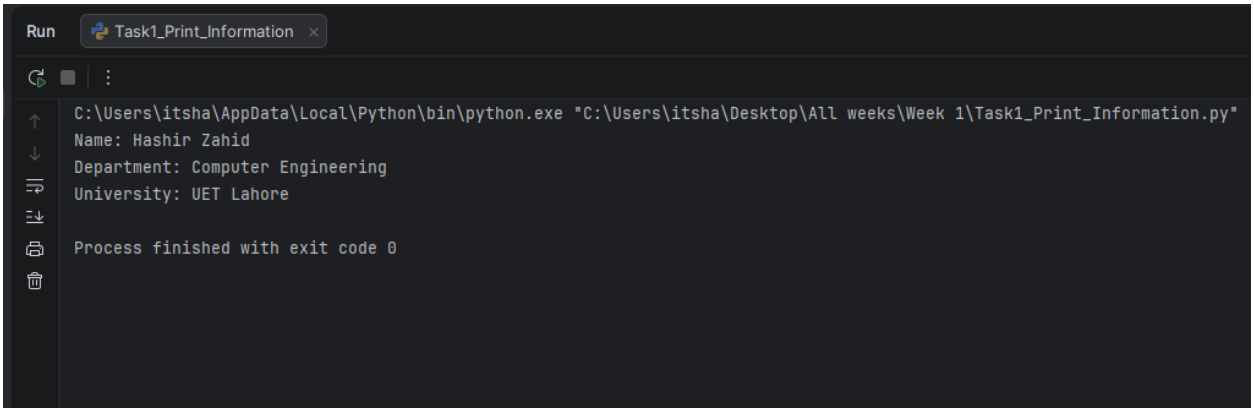
Code Summary:

This task demonstrates the basic usage of the print() function to display multiple lines of information. Each print() statement outputs a line of text containing personal details.

Key Code Lines:

```
# Task 1: Print Your Information
print("Name: Hashir Zahid")
print("Department: Computer Engineering")
print("University: UET Lahore")
```

Output:



Task 2: Using sep and end Parameters

Write a program that demonstrates the use of sep and end parameters in the print() function to customize output formatting.

Code Summary:

This task shows how to use sep to specify a custom separator between multiple arguments and end to control what appears at the end of the print statement.

Key Code Lines:

```
# Task 2: Using sep and end
print("UET","Lahore","Computer","Engineering", sep="-")
print("UET","Lahore", end="-")
print("Computer","Engineering")
```

Output:

Task 3: Format Output with Escape Sequences

Write a program that uses escape sequences (\n) to format output across multiple lines in a single print statement.

Code Summary:

This task demonstrates using the newline escape sequence (\n) to create multi-line output from a single print() call.

Key Code Lines:

```
# Task 3: Format Output
print("Name: Hashir\nAge: 19\nCity: Lahore")
```

Output:

Task 4: Take Name and Age Input

Write a program that takes user's name and age as input and displays them using type casting.

Code Summary:

This task demonstrates using the input() function with type casting. The name is kept as a string while the age is converted to an integer using int().

Key Code Lines:

```
# Task 4: Take Name and Age
name = str(input("Enter your Name:"))
age = int(input("Enter your age:"))
print("Name: ", name)
print("Your age: ", age)
```

Output:

Task 5: Identify Data Types

Write a program that declares variables of different data types and displays both their values and types using the type() function.

Code Summary:

This task demonstrates Python's built-in data types. Variables of type int, float, bool, and str are created, and the type() function is used to verify each variable's data type.

Key Code Lines:

```
# Task 5: Identify Data Types
x = 10
y = 3.14
z = True
name = "UET"
print(x)
print(y)
print(z)
print(type(x))
print(type(y))
print(type(z))
print(type(name))
```

Output:

Conclusions:

• Successfully understood and applied the print() function for output operations

• Learned to use sep and end parameters for custom formatting of output

• Mastered the use of escape sequences (\n) for multi-line formattin

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 2	**CLOs to be covered:** CLO1
Lab Title: For Loops	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 2

Lab Goals / Objectives:

• Understand the concept and syntax of for loops in Python

• Learn to use the range() function with for loops

• Apply for loops to solve practical problems

• Learn to iterate over strings using for loops

• Understand nested for loops and their applications

• Use for loops with conditional statements (if/else)

• Import and use the random and string modules

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. The for Loop in Python

A for loop in Python is used to iterate over a sequence (such as a list, tuple, string, or range) and execute a block of code for each item. Unlike other programming languages, Python's for loop is primarily designed for iteration over sequences.

```
for variable in sequence:
    # Code block to execute
```

2. The range() Function

The range() function generates a sequence of numbers. It is commonly used with for loops:

• range(stop) - Generates numbers from 0 to stop-1

• range(start, stop) - Generates numbers from start to stop-1

• range(start, stop, step) - Generates numbers from start to stop-1 with given step

```
for i in range(5):      # 0, 1, 2, 3, 4
for i in range(1, 6):   # 1, 2, 3, 4, 5
for i in range(0, 10, 2):  # 0, 2, 4, 6, 8
```

3. Iterating Over Strings

A string is a sequence of characters, so you can iterate over each character using a for loop:

```
for char in 'Hello':
    print(char)
```

4. The reversed() Function

The reversed() function returns a reversed iterator of a sequence:

```
for digit in reversed('1010'):
    print(digit)  # Prints: 0, 1, 0, 1
```

5. Nested For Loops

A nested for loop is a loop inside another loop. The inner loop executes completely for each iteration of the outer loop:

```
for i in range(3):
    for j in range(3):
        print(i, j)
```

6. The break Statement

The break statement is used to exit a loop prematurely when a certain condition is met:

```
for i in range(10):
    if i == 5:
        break
```

7. String Methods: isalpha()

The isalpha() method returns True if all characters in the string are alphabetic and there is at least one character:

```
'A'.isalpha()   # True
'1'.isalpha()   # False
```

8. The random and string Modules

The random module provides functions for generating random numbers and making random selections. The string module contains useful string constants:

• string.ascii_uppercase - All uppercase letters (A-Z)

• string.ascii_lowercase - All lowercase letters (a-z)

• string.digits - All digit characters (0-9)

• string.punctuation - All special characters

• random.choice(seq) - Returns a random element from a sequence

9. String Concatenation

Strings can be concatenated using the + operator:

```
result = ''
for i in range(3):
    result = result + str(i)
```

10. Prime Numbers

A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself. To check if a number is prime, we test divisibility from 2 up to number-1.

Lab Work:

Task 1: Binary to Decimal Converter

Write a program that converts a binary number into its decimal equivalent using a for loop.

Code Summary:

This program takes a binary string as input, iterates over each digit in reverse order using reversed(), multiplies each digit by 2 raised to the appropriate power, and accumulates the result to get the decimal equivalent.

Key Code Lines:

```
# LAB TASK 01: Binary to Decimal Converter
binary = input("Enter a binary number: ")
decimal = 0
power = 0
for digit in reversed(binary):
    decimal = decimal + int(digit) * (2 ** power)
    power = power + 1
print("Answer is:", decimal)
```

Output:

Task 2: Count Vowels and Consonants

Write a program that takes a sentence from the user and counts the number of vowels and consonants using a for loop.

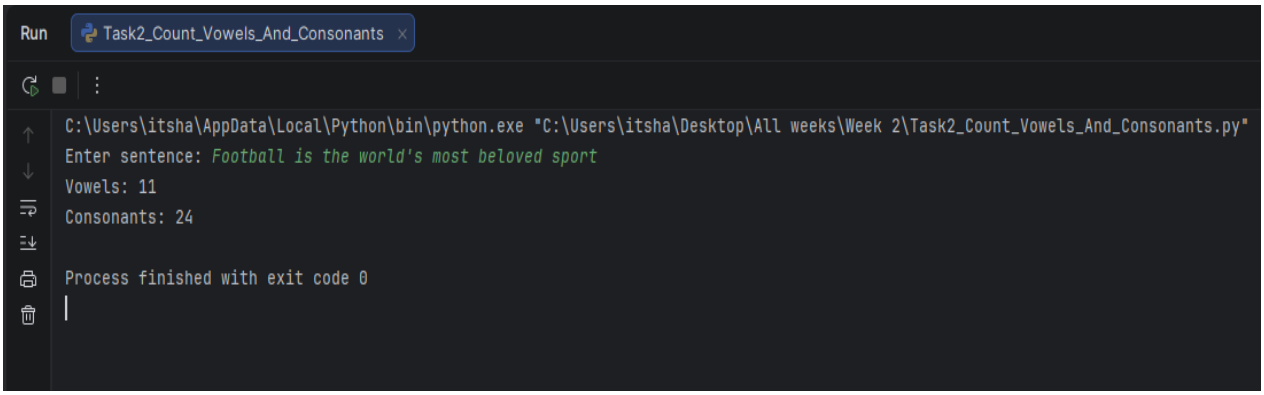
Code Summary:

This program iterates over each character in the input string, checks if it is an alphabetic character using isalpha(), then checks if it exists in the vowels string to classify it as a vowel or consonant.

Key Code Lines:

```
# LAB TASK 02: Count Vowels and Consonants
sentence = input("Enter sentence: ")
vowels = "aeiouAEIOU"
vowel_count = 0
consonant_count = 0
for letter in sentence:
    if letter.isalpha():
        if letter in vowels:
            vowel_count = vowel_count + 1
        else:
            consonant_count = consonant_count + 1
print("Vowels:", vowel_count)
print("Consonants:", consonant_count)
```

Output:



Task 3: Prime Numbers in Range

Write a program that takes a range from the user and prints all prime numbers within that range along with their sum.

Code Summary:

This program uses nested for loops. The outer loop iterates through numbers in the given range, and the inner loop checks divisibility from 2 to number-1. If no divisor is found, the number is prime.

Key Code Lines:

```
# LAB TASK 03: Prime Numbers in Range
start = int(input("Enter start: "))
end = int(input("Enter end: "))
total = 0
for number in range(start, end):
    if number > 1:
        prime = True
        for x in range(2, number):
            if number % x == 0:
                prime = False
                break
        if prime:
            print(number)
            total = total + number
print("Sum is:", total)
```

Output:

Task 4: Random Password Generator

Write a program that asks the user what kind of password they want and generates a random password of specified length.

Code Summary:

This program uses the random and string modules. It collects user preferences for character types (uppercase, lowercase, digits, special), builds a character pool, and randomly selects characters to generate a secure password.

Key Code Lines:

```
# LAB TASK 04: Random Password Generator
import random
import string
length = int(input("Enter length: "))
characters = ""
upper = input("Uppercase? y/n: ")
lower = input("Lowercase? y/n: ")
digit = input("Digits? y/n: ")
special = input("Special? y/n: ")
if upper == "y":
    characters = characters + string.ascii_uppercase
if lower == "y":
    characters = characters + string.ascii_lowercase
if digit == "y":
    characters = characters + string.digits
if special == "y":
    characters = characters + string.punctuation
password = ""
for p in range(length):
    password = password + random.choice(characters)
print("Password is:", password)
```

Output:

Task 5: Diamond Star Pattern

Write a program that prints a diamond-shaped star pattern using nested for loops.

Code Summary:

This program uses nested for loops to create a diamond pattern. The first outer loop prints an increasing triangle, and the second outer loop prints a decreasing triangle, creating a diamond shape.

Key Code Lines:

```
# LAB TASK 05: Diamond Star Pattern
rows = 4
for i in range(1, rows + 1):
    for j in range(i):
        print("*", end=" ")
    print()
for i in range(rows - 1, 0, -1):
    for j in range(i):
        print("*", end=" ")
    print()
```

Output:

Conclusions:

• Successfully understood and applied for loops for iterative problem solving

• Learned to use the range() function with various parameter combinations

• Mastered iterating over strings and using string methods like isalpha()

- Applied nested for loops to generate complex patterns
- Used the break statement to optimize loop execution
- Successfully imported and utilized the random and string modules
- Developed practical applications including number conversion and password generation

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 3	**CLOs to be covered:** CLO1
Lab Title: While Loops	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 3

Lab Goals / Objectives:

• Understand the concept and syntax of while loops in Python

• Differentiate between for loops and while loops

• Learn to use while loops for condition-based iteration

• Apply while loops with user-defined functions

• Understand function definition, parameters, and return values

• Use while loops for mathematical computations

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. The while Loop in Python

A while loop repeatedly executes a block of code as long as a given condition is True. It is used when the number of iterations is not known in advance.

```
while condition:
    # Code block to execute
```

The condition is evaluated before each iteration. If it is True, the loop body executes; if False, the loop terminates.

2. For Loop vs While Loop

• For loop: Used when the number of iterations is known or when iterating over a sequence

• While loop: Used when the number of iterations depends on a condition

3. User-Defined Functions

A function is a reusable block of code that performs a specific task. Functions are defined using the def keyword:

```
def function_name(parameters):
    # Function body
    return value
```

Functions can accept parameters (arguments) and return values using the return statement.

4. Function Parameters and Arguments

• Parameters are variables listed in the function definition

• Arguments are the actual values passed to the function when called

• Functions can have multiple parameters separated by commas

5. The return Statement

The return statement exits a function and optionally passes a value back to the caller:

```
def add(a, b):
    return a + b
```

6. Factorial Calculation

The factorial of a number n (denoted as n!) is the product of all positive integers less than or equal to n:

```
n! = 1 x 2 x 3 x ... x n
```

7. Permutation and Combination

Permutation P(n,r) = n! / (n-r)! represents the number of ways to arrange r items from n items.

Combination C(n,r) = n! / (r! x (n-r)!) represents the number of ways to choose r items from n items without regard to order.

8. Lambda Functions

A lambda function is a small anonymous function defined with the lambda keyword. It can take any number of arguments but has only one expression:

```
lambda arguments : expression
```

Lambda functions are often used for simple operations that can be written in a single line.

9. Temperature Conversion Formulas

Celsius to Fahrenheit: F = (C x 9/5) + 32

Fahrenheit to Celsius: C = (F - 32) x 5/9

10. GPA Calculation

GPA (Grade Point Average) is calculated by summing the product of grade points and credit hours for all subjects, then dividing by the total credit hours:

GPA = Sum(Grade Point x Credit Hours) / Total Credit Hours

Lab Work:

Task 1: Print Hello 5 Times

Write a program that uses a while loop to print "Hello" five times.

Code Summary:

This task demonstrates the basic structure of a while loop: a counter variable is initialized, checked against a condition, and incremented on each pass until the condition becomes false.

Key Code Lines:

```
# Task 1: Print Hello 5 Times
count = 1
while count <= 5:
    print("Hello")
    count += 1
```

Output:

Task 2: Print Numbers 1 to 5

Write a program that uses a while loop to print numbers from 1 to 5.

Code Summary:

This task uses a while loop with a counter variable that is printed and incremented on each iteration until it exceeds 5.

Key Code Lines:

```
# Task 2: Print Numbers 1 to 5
num = 1
while num <= 5:
    print(num)
    num += 1
```

Output:

Task 3: Row of Stars

Write a program that uses a while loop to print a fixed row of stars, five times.

Code Summary:

This task repeats a single line of five stars using a while loop controlled by a row counter.

Key Code Lines:

```
# Task 3: Row of Stars
rows = 5
i = 1
while i <= rows:
    print("* * * * *")
    i += 1
```

Output:

Task 4: Increasing Star Pattern

Write a program that uses a while loop to print an increasing star pattern (1 star, 2 stars, 3 stars...).

Code Summary:

This task multiplies the "*" character by the current counter value on each iteration, producing a triangle of stars using only a while loop.

Key Code Lines:

```
# Task 4: Increasing Star Pattern
rows = 5
i = 1
while i <= rows:
    print("*" * i)
    i += 1
```

Output:

Task 5: Repeated Digit Pattern

Write a program that uses a while loop to print each number repeated according to its own value (e.g. 1, 22, 333, ...).

Code Summary:

This task converts the counter to a string and repeats it by its own value on each while-loop pass, building a numeric pattern.

Key Code Lines:

```
# Task 5: Repeated Digit Pattern
i = 1
while i <= 5:
```

```
    print(i * str(i))
    i += 1
```

Output:

Conclusions:

• Successfully understood and applied while loops for condition-based iteration

• Learned to define and call user-defined functions with parameters and return values

• Mastered the use of lambda functions for simple operations

• Applied mathematical concepts through programming (factorial, permutation, combination)

• Developed practical utility programs (temperature converter, GPA calculator)

• Understood the difference between for loops and while loops

• Gained experience in modular programming through function decomposition

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 4	**CLOs to be covered:** CLO1
Lab Title: Nested Loops	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 4

Lab Goals / Objectives:

• Understand the concept of nested loops (loops within loops)

• Learn different types of function arguments: positional, keyword, default

• Understand variable-length arguments (*args)

• Apply nested loops to create geometric patterns

• Master function definition with multiple parameter types

• Learn to return multiple values and use them effectively

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. Nested Loops

A nested loop is a loop inside the body of another loop. The inner loop executes completely for each iteration of the outer loop. Both for loops and while loops can be nested.

```
for i in range(3):
    for j in range(3):
        print(i, j)
```

In this example, the inner loop runs 3 times for each of the 3 iterations of the outer loop, resulting in 9 total iterations.

2. Using Nested Loops for Patterns

Nested loops are commonly used to generate geometric patterns. The outer loop typically controls the rows, and the inner loop controls the columns or elements within each row.

3. Positional Arguments

Positional arguments are passed to a function in the order they are defined. The position of the argument determines which parameter it corresponds to:

```
def greet(name, age):
    print(name, age)
greet('Ali', 20)  # name='Ali', age=20
```

4. Keyword Arguments

Keyword arguments are passed by explicitly specifying the parameter name. This allows arguments to be passed in any order:

```
def greet(name, age):
    print(name, age)
greet(age=20, name='Ali')
```

5. Default Arguments

Default arguments have a default value specified in the function definition. If the caller does not provide a value, the default is used:

```
def power(base, exponent=2):
    return base ** exponent
print(power(5))      # 25 (5^2)
print(power(2, 3))   # 8 (2^3)
```

6. Variable-Length Arguments (*args)

The *args syntax allows a function to accept any number of positional arguments. These arguments are collected into a tuple:

```
def total(*numbers):
    return sum(numbers)
print(total(1, 2, 3, 4))  # 10
```

7. The f-string Formatting

f-strings provide an elegant way to embed expressions inside string literals for formatted output:

```
name = 'Ali'
age = 20
print(f'Name: {name}, Age: {age}')
```

8. The sum() and len() Built-in Functions

• sum(iterable) - Returns the sum of all items in an iterable

• len(object) - Returns the number of items in an object

```
numbers = [1, 2, 3, 4]
print(sum(numbers))  # 10
print(len(numbers))  # 4
```

9. Arithmetic Operators

• ** (Exponent): Raises the left operand to the power of the right

• % (Modulus): Returns the remainder of division

• // (Floor Division): Returns the integer part of division

10. Comparison Operators

• > (Greater than), < (Less than)

• >= (Greater than or equal), <= (Less than or equal)

• == (Equal to), != (Not equal to)

• and (Logical AND), or (Logical OR)

Lab Work:

Task 1: Nested Multiplication Table (While)

Write a program that uses nested while loops to print a multiplication table for numbers 1 to 3, each up to 10.

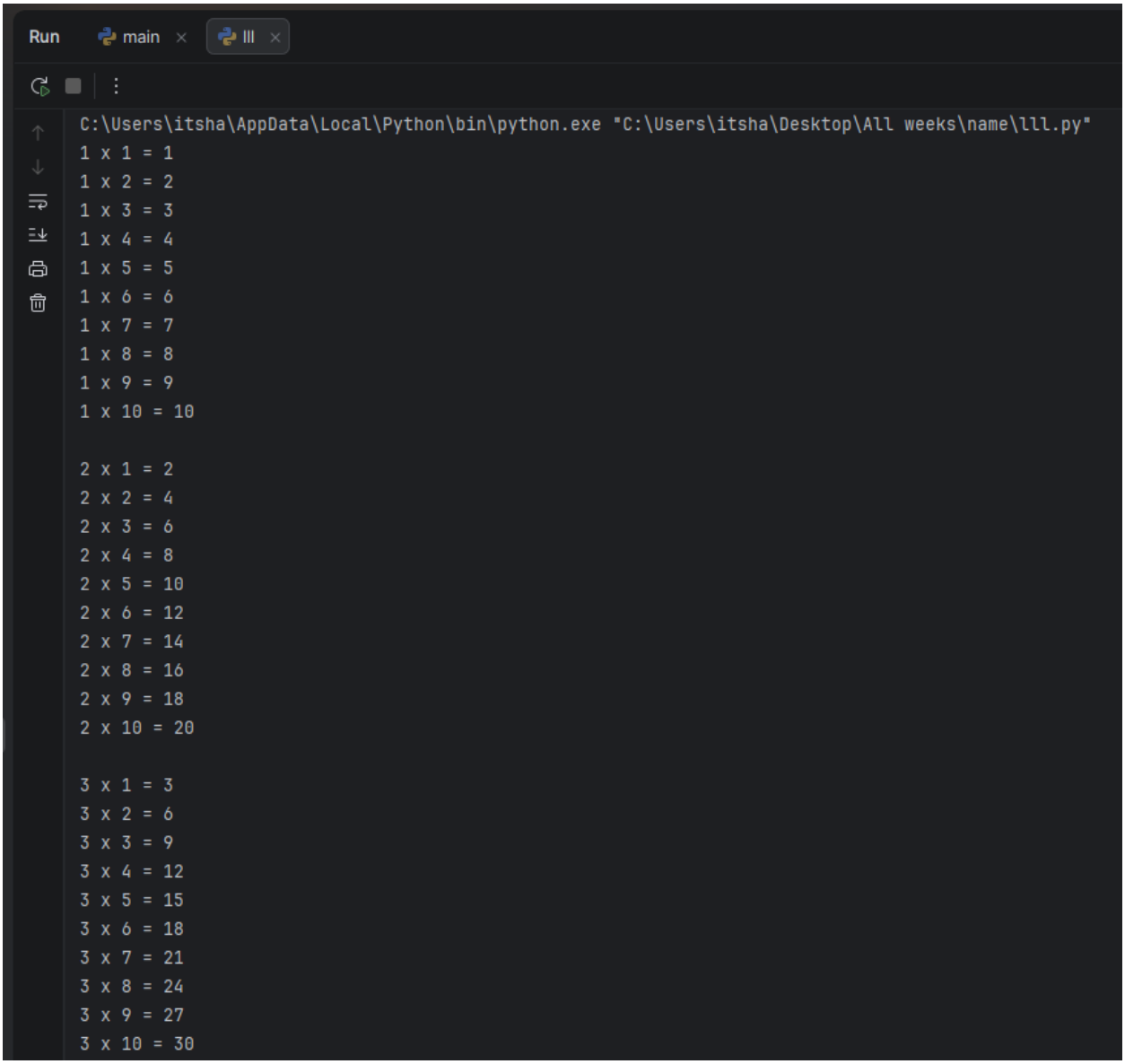
Code Summary:

The outer while loop controls the row number (1 to 3) and the inner while loop controls the column number (1 to 10), printing each row's multiplication results before moving to the next row.

Key Code Lines:

```
# Task 1: Nested Multiplication Table
i = 1
while i <= 3:
    j = 1
    while j <= 10:
        print(i, "x", j, "=", i * j)
        j += 1
    print()
    i += 1
```

Output:



Task 2: Decreasing Number Triangle (While)

Write a program that uses nested while loops to print a decreasing triangle of numbers.

Code Summary:

The outer while loop counts down the number of rows, while the inner while loop prints numbers from 1 up to the current row count, forming a shrinking triangle.

Key Code Lines:

```
# Task 2: Decreasing Number Triangle
i = 5
while i >= 1:
    j = 1
    while j <= i:
        print(j, end=" ")
        j += 1
```

```
    print()
    i -= 1
```

Output:

Task 3: Count Vowels and Consonants (Nested For)

Write a program that counts the vowels and consonants in a sentence using a nested for loop.

Code Summary:

The outer for loop scans each character of the sentence, and the inner for loop checks that character against every vowel; a for-else construct increments the consonant count if no vowel match is found.

Key Code Lines:

```
# Task 3: Count Vowels and Consonants
sentence = input("Enter a sentence: ")

vowels = 0
consonants = 0

for ch in sentence:
    if ('a' <= ch <= 'z') or ('A' <= ch <= 'Z'):
        for v in "aeiouAEIOU":
            if ch == v:
                vowels += 1
                break
        else:
            consonants += 1

print("Number of vowels:", vowels)
print("Number of consonants:", consonants)
```

Output:

Task 4: Increasing and Decreasing Star Pyramid

Write a program that uses nested for loops to print an increasing star pattern followed by a decreasing star pattern.

Code Summary:

Two separate nested for-loop blocks are used: the first builds rows of stars that grow in size, and the second builds rows that shrink, together forming a simple pyramid effect.

Key Code Lines:

```
# Task 4: Increasing and Decreasing Star Pattern
for i in range(1, 5):
    for j in range(i):
        print("*", end=" ")
    print()

for i in range(3, 0, -1):
    for j in range(i):
        print("*", end=" ")
    print()
```

Output:

Conclusions:

- Successfully understood and applied nested loops for pattern generation
- Mastered different types of function arguments: positional, keyword, and default
- Learned to use variable-length arguments (*args) for flexible function calls
- Applied conditional statements within functions for decision-making
- Understood the concept of returning values from functions
- Developed multiple utility functions demonstrating various function concepts
- Gained practical experience in modular and reusable code design

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 5	**CLOs to be covered:** CLO2
Lab Title: Functions	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO2	Design and implement modular programs using functions, recursion, and reusable code constructs.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 5

Lab Goals / Objectives:

• Understand the concept and importance of functions in Python

• Learn all Python built-in functions with practical examples

• Understand function definition, parameters, and return values

• Master different types of function arguments

• Learn about recursive functions and their applications

• Understand the scope and lifetime of variables in functions

• Apply functions to solve real-world problems

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. What is a Function?

A function is a reusable block of code that performs a specific task. Functions help break down large programs into smaller, manageable pieces. They promote code reuse and make programs more organized and easier to maintain.

2. Types of Functions in Python

Python provides two types of functions:

• Built-in Functions: Pre-defined functions available in Python (e.g., print(), input(), len(), type())

• User-defined Functions: Functions created by the programmer using the def keyword

3. Function Definition Syntax

Functions are defined using the def keyword followed by the function name and parentheses containing optional parameters:

```
def function_name(parameters):
    """Docstring (optional)"""
    # Function body
    return value  # Optional
```

4. Built-in Functions in Python

4.1 print() Function

The print() function outputs the specified message to the console. It can take multiple arguments and supports various formatting options through sep and end parameters.

```
print("Hello, World!")
print("Name:", "Ali", sep="-")
```

4.2 input() Function

The input() function reads a line from input, converts it to a string, and returns it. It optionally takes a prompt message as an argument.

```
name = input("Enter your name: ")
```

4.3 type() Function

The type() function returns the data type of the specified object. It is useful for debugging and type checking.

```
type(10)      # <class 'int'>
type(3.14)    # <class 'float'>
type("Hello") # <class 'str'>
```

4.4 int(), float(), str() Functions

These functions convert values to integer, floating-point, and string types respectively:

```
int("10")      # Converts string to integer
float("3.14")  # Converts string to float
str(100)       # Converts integer to string
```

4.5 len() Function

The len() function returns the number of items in an object such as a string, list, tuple, or dictionary:

```
len("Hello")   # Returns 5
len([1,2,3])   # Returns 3
```

4.6 max() and min() Functions

The max() function returns the largest item in an iterable or the largest of two or more arguments. The min() function returns the smallest.

```
max(10, 25, 7)  # Returns 25
min(10, 25, 7)  # Returns 7
```

4.7 sum() Function

The sum() function adds all items in an iterable and returns the total:

```
sum([1, 2, 3, 4])  # Returns 10
```

4.8 range() Function

The range() function generates a sequence of numbers. It is commonly used with for loops:

```
range(5)        # 0, 1, 2, 3, 4
range(1, 6)     # 1, 2, 3, 4, 5
range(0, 10, 2) # 0, 2, 4, 6, 8
```

4.9 abs() Function

The abs() function returns the absolute value of a number:

```
abs(-10)   # Returns 10
abs(3.14)  # Returns 3.14
```

4.10 round() Function

The round() function rounds a number to a given precision in decimal digits:

```
round(3.14159, 2)  # Returns 3.14
```

4.11 pow() Function

The pow() function returns the value of x to the power of y (x^y). It is equivalent to the ** operator:

```
pow(2, 3)   # Returns 8
```

4.12 sorted() Function

The sorted() function returns a new sorted list from the elements of any iterable:

```
sorted([3, 1, 4, 1, 5])  # Returns [1, 1, 3, 4, 5]
```

5. Function Arguments

5.1 Positional Arguments

Arguments passed in the order they are defined. The position determines which parameter receives the value.

5.2 Keyword Arguments

Arguments passed with explicit parameter names, allowing them to be in any order.

5.3 Default Arguments

Parameters that have default values. If the caller does not provide a value, the default is used.

5.4 Variable-Length Arguments (*args)

Allows a function to accept any number of positional arguments. The arguments are collected into a tuple.

6. The return Statement

The return statement exits a function and passes a value back to the caller. A function can return multiple values as a tuple.

7. Docstrings

Docstrings are string literals that appear as the first statement in a function. They document what the function does and can be accessed via the __doc__ attribute.

```
def greet():
    """This function prints a greeting message."""
    print("Hello!")
```

Lab Work:

Task 1: Built-in Functions Demonstration

Demonstrate the use of various Python built-in functions with practical examples.

Code Summary:

This task demonstrates essential built-in functions including type() for checking data types, len() for finding length, max() and min() for comparisons, and abs() for absolute value.

Key Code Lines:

```
# Demonstration of Built-in Functions
print("Type of 10:", type(10))
print("Length of 'Hello':", len("Hello"))
print("Max of 10, 25:", max(10, 25))
print("Min of 10, 25:", min(10, 25))
print("Absolute of -15:", abs(-15))
print("Power 2^3:", pow(2, 3))
```

Output:

Task 2: Type Casting Functions

Demonstrate type casting using int(), float(), and str() functions.

Code Summary:

This task shows how to convert between different data types using built-in casting functions, which is essential for handling user input and performing calculations.

Key Code Lines:

```
# Type Casting Demonstration
x = "100"
y = int(x)
print("String to int:", y, type(y))

a = 3.14
b = str(a)
print("Float to string:", b, type(b))

c = 10
d = float(c)
print("Int to float:", d, type(d))
```

Output:

Task 3: Greet Function

Define and call a simple function that prints a welcome message.

Code Summary:

This task demonstrates the basic structure of a user-defined function: definition using def, function body with statements, and calling the function by name.

Key Code Lines:

```
# Simple Greet Function
def greet():
    print("Welcome to Python Programming")

greet()
greet()
```

Output:

Task 4: Function with Parameters and Return

Define a function that takes two numbers as parameters and returns their sum.

Code Summary:

This task demonstrates passing parameters to a function and returning a computed result using the return statement.

Key Code Lines:

```
# Function with Parameters and Return
def addNumbers(a, b):
    Total = a + b
    return Total
```

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
print("Sum =", addNumbers(num1, num2))
```

Output:

Task 5: Function with Default Argument

Define a function power(base, exponent=2) that calculates power with a default exponent of 2.

Code Summary:

This task demonstrates default parameter values. When the second argument is omitted, the default value is used.

Key Code Lines:

```
# Function with Default Argument
def power(base, exponent=2):
    return base ** exponent

print("Square of 5:", power(5))
print("5 to power 3:", power(5, 3))
```

Output:

Conclusions:

- Successfully understood the importance and working of functions in Python
- Learned and demonstrated all major Python built-in functions
- Mastered function definition with def keyword and various argument types
- Understood positional, keyword, default, and variable-length arguments
- Learned to use the return statement for passing values back to callers
- Understood the concept of docstrings for function documentation
- Applied functions to write cleaner, reusable, and modular code

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 6	**CLOs to be covered:** CLO2
Lab Title: Local and Global Variables	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO2	Design and implement modular programs using functions, recursion, and reusable code constructs.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 6

Lab Goals / Objectives:

• Understand the concept of variable scope in Python

• Differentiate between local and global variables

• Learn how to access global variables inside functions

• Understand the global keyword and its usage

• Learn about the globals() and locals() functions

• Understand variable lifetime and memory management

• Apply scope concepts to write better-structured programs

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. What is Variable Scope?

Variable scope defines the region of the program where a variable is accessible. In Python, variables can have either local or global scope depending on where they are declared.

2. Local Variables

A local variable is declared inside a function and is only accessible within that function. It is created when the function is called and destroyed when the function completes execution. Local variables cannot be accessed from outside the function.

```
def my_function():
    x = 10  # Local variable
    print(x)
my_function()
# print(x)  # Error: x is not defined outside the function
```

3. Global Variables

A global variable is declared outside all functions, typically at the top level of the program. It is accessible from any part of the program, including inside functions (for reading). Global variables exist throughout the program's execution.

```
x = 10  # Global variable
def my_function():
    print(x)  # Can read global variable
my_function()
```

4. The global Keyword

The global keyword is used inside a function to declare that a variable refers to the global variable, not a local one. This allows functions to modify global variables.

```
count = 0
def increment():
    global count
```

```
    count = count + 1
increment()
print(count)  # Output: 1
```

5. globals() Function

The globals() function returns a dictionary containing all global variables and their values. It can be used to access global variables by name from within a function.

```
x = 40
def show():
    x = 10
    print(x)           # Prints local x: 10
    print(globals()["x"])  # Prints global x: 40
```

6. locals() Function

The locals() function returns a dictionary containing all local variables and their values in the current scope. It is useful for debugging and introspection.

```
def example():
    a = 1
    b = 2
    print(locals())  # {'a': 1, 'b': 2}
```

7. Variable Shadowing

When a local variable has the same name as a global variable, the local variable 'shadows' the global variable inside the function. Within the function, only the local variable is accessible.

```
x = 20
def func():
    x = 5  # Local x shadows global x
    print(x)  # Prints 5
func()
print(x)  # Prints 20 (global x unchanged)
```

8. The nonlocal Keyword

The nonlocal keyword is used in nested functions to refer to a variable in the nearest enclosing scope (excluding global). It allows inner functions to modify variables from the outer function.

```
def outer():
    x = 10
    def inner():
        nonlocal x
        x = 20
    inner()
    print(x)  # Output: 20
```

9. Best Practices for Variable Scope

• Minimize the use of global variables to avoid unintended side effects

• Use function parameters and return values to pass data between functions

• Use meaningful variable names to avoid naming conflicts

• Document which variables are global when they must be used

• Prefer local variables for temporary calculations

Lab Work:

Task 1: Local vs Global Variable Demonstration

Write a program that demonstrates the difference between local and global variables.

Code Summary:

This task demonstrates how a local variable inside a function shadows the global variable of the same name. The globals() function is used to access the global variable even when shadowed.

Key Code Lines:

```
# Local vs Global Variable Demonstration
x = "45"         # Global variable x
y = "    "       # Global variable y (whitespace)

def welcome():
    x = 40       # Local variable x (shadows global x)
```

```
    print("Local x:", x)         # Prints local: 40
    print("Global x:", globals()["x"])  # Prints global: 45

welcome()
```

Output:

Task 2: Using the global Keyword

Code Summary:

This task demonstrates how to use the global keyword to modify a global variable from within a function. Without global, the assignment would create a new local variable.

Key Code Lines:

```
# Using global Keyword
counter = 0

def increment():
    global counter
    counter = counter + 1
    print("Counter inside function:", counter)

increment()
increment()
print("Counter outside function:", counter)
```

Output:

Task 3: globals() and locals() Functions

Write a program that demonstrates the use of globals() and locals() built-in functions.

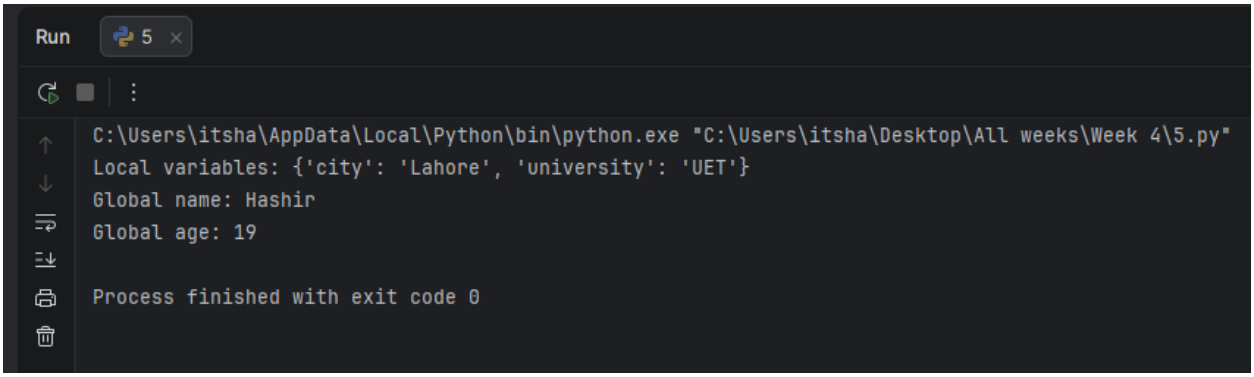
Code Summary:

This task shows how to inspect the global and local namespaces using the globals() and locals() functions, which is useful for debugging and understanding variable scope.

Key Code Lines:

```
# globals() and locals() Demonstration
name = "Hashir"
age = 19

def show_scope():
    city = "Lahore"
    university = "UET"
    print("Local variables:", locals())
    print("Global name:", globals()["name"])
    print("Global age:", globals()["age"])

show_scope()
```

Output:



Task 4: Nested Functions with nonlocal

Write a program that uses nested functions and the nonlocal keyword.

Code Summary:

This task demonstrates how the nonlocal keyword allows an inner function to modify a variable from its enclosing (non-global) scope.

Key Code Lines:

```
# Nested Functions with nonlocal
def outer_function():
    message = "Hello"

    def inner_function():
        nonlocal message
        message = "Hello, World!"

    print("Before:", message)
    inner_function()
    print("After:", message)

outer_function()
```

Output:

Task 5: Practical Scope Exercise

Write a program that calculates the total price including tax using global variables for the tax rate.

Code Summary:

This task demonstrates a practical use case for global variables where the tax rate is defined globally and accessed within a function to perform calculations.

Key Code Lines:

```
# Practical Scope Exercise
TAX_RATE = 0.17  # Global tax rate

def calculate_total(price):
    tax = price * TAX_RATE
    total = price + tax
    return total
```

```
price1 = float(input("Enter price: "))
print("Total with tax:", calculate_total(price1))
```

Output:

Conclusions:

• Successfully understood the concept of variable scope in Python

• Differentiated between local variables (function scope) and global variables (module scope)

• Learned to use the global keyword to modify global variables inside functions

• Understood variable shadowing and how to access global variables using globals()

• Learned about the nonlocal keyword for nested function scopes

• Applied scope concepts to write well-structured and maintainable programs

• Understood best practices for minimizing global variable usage

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 7	**CLOs to be covered:** CLO3
Lab Title: List, Tuple	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO3	Use appropriate data structures to organize, store, and manipulate data efficiently.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 7

Lab Goals / Objectives:

• Understand the concept of sequences in Python

• Learn to create, access, and manipulate lists

• Learn to create, access, and work with tuples

• Understand the differences between lists and tuples

• Master list methods and operations

• Learn list slicing and indexing techniques

• Apply lists and tuples to solve practical problems

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. What are Sequences?

Sequences are ordered collections of items in Python. Each item in a sequence has an index (position) starting from 0. Python provides several built-in sequence types including lists, tuples, and strings.

2. Lists in Python

A list is a mutable (changeable) ordered sequence of items. Lists are defined using square brackets [] and can contain items of different data types. Lists are one of the most versatile and commonly used data structures in Python.

```
my_list = [1, 2, 3, "hello", 3.14]
```

3. Creating Lists

Lists can be created in several ways:

• Using square brackets: `my_list = [1, 2, 3]`

• Using the list() constructor: `my_list = list((1, 2, 3))`

• Using list comprehension: `my_list = [x for x in range(5)]`

• Empty list: `my_list = []`

4. Accessing List Elements

List elements are accessed using indexing. Python uses zero-based indexing (first element is at index 0). Negative indexing starts from the end (-1 is the last element).

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])   # apple
print(fruits[-1])  # cherry
```

5. List Slicing

Slicing extracts a portion of a list using the syntax list[start:end:step]. The end index is exclusive.

```
numbers = [0, 1, 2, 3, 4, 5]
print(numbers[1:4])   # [1, 2, 3]
print(numbers[:3])    # [0, 1, 2]
print(numbers[::2])   # [0, 2, 4]
```

6. List Methods

Python lists provide many built-in methods:

• append(item) - Adds an item to the end of the list

• insert(index, item) - Inserts an item at a specific position

• remove(item) - Removes the first occurrence of an item

• pop(index) - Removes and returns the item at the given index

• sort() - Sorts the list in ascending order

• reverse() - Reverses the order of the list

• index(item) - Returns the index of the first occurrence

• count(item) - Returns the number of occurrences

• extend(iterable) - Adds all items from an iterable to the list

• clear() - Removes all items from the list

7. List Operations

• Concatenation (+): Combines two lists

• Repetition (*): Repeats a list

• Membership (in): Checks if an item exists in the list

• Length (len()): Returns the number of items

8. Tuples in Python

A tuple is an immutable (unchangeable) ordered sequence of items. Tuples are defined using parentheses () or just commas. Once created, tuple elements cannot be modified, added, or removed.

```
my_tuple = (1, 2, 3, "hello")
```

9. Why Use Tuples?

• Tuples are faster than lists due to immutability

• Tuples protect data that should not be modified

• Tuples can be used as dictionary keys (lists cannot)

• Tuples are useful for representing fixed collections of items

10. Tuple Operations

Tuples support indexing, slicing, concatenation, repetition, and membership testing just like lists. However, tuples do not support methods that modify the tuple (no append, remove, etc.).

11. Unpacking

Both lists and tuples support unpacking, where elements are assigned to individual variables:

```
coordinates = (10, 20)
x, y = coordinates
```

12. List vs Tuple

| Feature | List | Tuple |

| Mutable | Yes | No |

| Syntax | [] | () |

| Methods | Many | Few |

| Speed | Slower | Faster |

| Use Case | Dynamic data | Fixed data |

Lab Work:

Task 1: Append to a List

Write a program that creates a list of fruits and appends a new item to it.

Code Summary:

This task demonstrates list mutability using the append() method, which adds a new element to the end of an existing list.

Key Code Lines:

```
# Task 1: Append to a List
fruits = ["Apple", "Banana"]
fruits.append("Orange")
print(fruits)
```

Output:

Task 2: Count List Items with a Function

Write a function that returns the number of items in a list, and test it with a list of fruits.

Code Summary:

This task defines a function that wraps Python's built-in len() to count the elements of any list passed to it.

Key Code Lines:

```
# Task 2: Count List Items
def count_items(items):
    return len(items)

fruits = ["Apple", "Banana", "Mango"]
print("Total Items =", count_items(fruits))
```

Output:

Task 3: Search for a Name in a List

Write a program that checks whether a student name entered by the user exists in a list of students.

Code Summary:

This task uses the in operator to search a list for a matching value and reports whether the student is present or absent.

Key Code Lines:

```
# Task 3: Search for a Name in a List
students = ["Ali", "Hamza", "Ahmed", "Usman"]
name = input("Enter student name: ")

if name in students:
    print("Present")
else:
    print("Absent")
```

Output:

Task 4: Access Tuple Elements

Write a program that stores employee details in a tuple and accesses each field by index.

Code Summary:

This task demonstrates tuple indexing, showing that a tuple's fixed, ordered elements can be retrieved using their position.

Key Code Lines:

```
# Task 4: Access Tuple Elements
employee = ("E101", "Hamza", "Manage", 50000)
print("Employee ID:", employee[0])
print("Name:", employee[1])
print("Job:", employee[2])
print("Salary:", employee[3])
```

Output:

Task 5: Sum of List Values

Write a program that stores item prices in a list and calculates the total bill.

Code Summary:

This task uses the built-in sum() function to add up all the numeric values stored in a list.

Key Code Lines:

```
# Task 5: Sum of List Values
prices = [120, 80, 250, 60]
print("Items Prices:", prices)
print("Total Bill =", sum(prices))
```

Output:

Task 6: Check Book Availability

Write a program that checks whether a book entered by the user is available in a list of books.

Code Summary:

This task again uses the in operator, this time over a list of book titles, to determine availability.

Key Code Lines:

```
# Task 6: Check Book Availability
books = ["Python", "Java", "C++", "Database"]
book = input("Enter book name: ")

if book in books:
    print("Book Available")
else:
    print("Book Not Available")
```

Output:

Conclusions:

• Successfully understood and created lists and tuples in Python

• Mastered list indexing, slicing, and various list methods

- Learned list comprehension for concise list creation
- Understood the immutability of tuples and their advantages
- Applied tuple unpacking for elegant variable assignment
- Worked with nested lists for multi-dimensional data
- Understood when to use lists (mutable data) vs tuples (immutable data)

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 8	**CLOs to be covered:** CLO3
Lab Title: Sets, Dictionary	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO3	Use appropriate data structures to organize, store, and manipulate data efficiently.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 8

Lab Goals / Objectives:

• Understand the concept of sets and their properties

• Learn to create, modify, and perform operations on sets

• Understand the concept of dictionaries as key-value pairs

• Learn to create, access, and manipulate dictionaries

• Master set operations: union, intersection, difference

• Understand dictionary methods and iteration techniques

• Apply sets and dictionaries to practical problems

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. Sets in Python

A set is an unordered collection of unique elements. Sets are mutable but can only contain immutable (hashable) elements. Sets are defined using curly braces {} or the set() constructor.

```
my_set = {1, 2, 3, 4, 5}
```

Key properties of sets:

• Unordered: Elements have no defined order

• Unique: Duplicate elements are automatically removed

• Mutable: Elements can be added and removed

• Elements must be immutable (numbers, strings, tuples)

2. Creating Sets

• Using curly braces: `s = {1, 2, 3}`

• Using set() constructor: `s = set([1, 2, 3])`

• Empty set: `s = set()` (Note: {} creates an empty dictionary)

3. Set Methods

• add(item) - Adds a single item to the set

• update(iterable) - Adds multiple items from an iterable

• remove(item) - Removes an item (raises error if not found)

• discard(item) - Removes an item (no error if not found)

• pop() - Removes and returns an arbitrary item

• clear() - Removes all items from the set

4. Set Operations

• Union (| or union()): Combines elements from both sets

• Intersection (& or intersection()): Elements common to both sets

• Difference (- or difference()): Elements in first set but not second

• Symmetric Difference (^): Elements in either set but not both

• Subset (<= or issubset()): Checks if all elements are in another set

• Superset (>= or issuperset()): Checks if set contains all elements of another

5. Dictionaries in Python

A dictionary is an unordered collection of key-value pairs. Each key is unique and maps to a value. Dictionaries are defined using curly braces {} with key:value syntax.

```
my_dict = {"name": "Ali", "age": 20, "city": "Lahore"}
```

Key properties of dictionaries:

• Keys must be unique and immutable (strings, numbers, tuples)

• Values can be any data type

• Dictionaries are mutable

• Dictionaries are optimized for fast key lookup

6. Accessing Dictionary Elements

Values are accessed using their keys:

```
student = {"name": "Ali", "age": 20}
print(student["name"])   # Ali
print(student.get("age"))  # 20
```

7. Dictionary Methods

• keys() - Returns all keys in the dictionary

• values() - Returns all values in the dictionary

• items() - Returns all key-value pairs as tuples

• get(key, default) - Returns value for key, or default if not found

• update(dict) - Merges another dictionary

• pop(key) - Removes and returns the value for the given key

• popitem() - Removes and returns the last inserted key-value pair

• setdefault(key, value) - Sets a key if it does not exist

• clear() - Removes all items

8. Iterating Over Dictionaries

```
student = {"name": "Ali", "age": 20}
for key in student:
    print(key, student[key])

for key, value in student.items():
    print(key, value)
```

9. Nested Dictionaries

Dictionaries can contain other dictionaries as values, allowing representation of complex data structures:

```
students = {
    "student1": {"name": "Ali", "age": 20},
    "student2": {"name": "Ahmed", "age": 21}
}
```

10. When to Use Sets vs Dictionaries

• Use sets when you need an unordered collection of unique items and need to perform set operations

• Use dictionaries when you need to associate values with keys for fast lookup

Lab Work:

Task 1: Set Update and Union

Write a program that combines two sets using update() and union(), and prints the results.

Code Summary:

This task shows update(), which merges another set's elements into the first set in place, and union(), which returns a new combined set without modifying either original set.

Key Code Lines:

```
# Task 1: Set Update and Union
s1 = {"Ali", "Bilal", "Zain", "Ahmad"}
s2 = {45, 47, 28, 49}
s1.update(s2)
print(s1)
print(s2)
z = s1.union(s1)
print(z)
```

Output:

Task 2: Set Intersection

Write a program that finds the common elements between two sets using intersection().

Code Summary:

This task demonstrates intersection(), which returns only the elements that are present in both sets being compared.

Key Code Lines:

```
# Task 2: Set Intersection
s1 = {1, 2, 3}
s2 = {"UET", "CS", "CE", 2}
s3 = {7, 5, 9}
c = s1.intersection(s2)
print(c)
```

Output:

Task 3: Dictionary Lookup by Key

Write a program that stores student marks in a dictionary and looks up a mark by student name.

Code Summary:

This task demonstrates checking for key existence with the in operator before retrieving the associated value from a dictionary.

Key Code Lines:

```
# Task 3: Dictionary Lookup by Key
marks = {
    "Ali": 85,
    "Hamza": 90,
    "Ahmed": 78
}
name = input("Enter student name: ")

if name in marks:
    print("Marks =", marks[name])
else:
    print("Student Not Found")
```

Output:

Task 4: Dictionary keys() and values()

Write a program that creates a dictionary of student details and displays its keys and values separately.

Code Summary:

This task demonstrates the keys() and values() dictionary methods, which return view objects of a dictionary's keys and values respectively.

Key Code Lines:

```
# Task 4: Dictionary keys() and values()
student = {"Name": "Hamza", "Age": 20}
print(student.keys())

student = {"Name": "Hamza", "Age": 20}
print(student.values())
```

Output:

Conclusions:

• Successfully understood and created sets and dictionaries in Python

• Mastered set operations: union, intersection, difference, and symmetric difference

• Learned set methods for adding, removing, and querying elements

• Understood dictionary structure as key-value pairs

• Mastered dictionary access, modification, and iteration techniques

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 9	**CLOs to be covered:** CLO1
Lab Title: Viva	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**

Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 9

Lab Goals / Objectives:

• Review and reinforce concepts from Weeks 1-8

• Demonstrate proficiency in loops and conditional statements

• Apply nested loops for complex pattern generation

• Practice dictionary creation and iteration

• Solve problems combining multiple programming concepts

• Prepare for oral examination on programming fundamentals

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. Comprehensive Review of Python Fundamentals

This lab serves as a comprehensive review of all concepts covered in the first eight weeks. Students demonstrate their understanding through practical problem-solving tasks that combine multiple programming concepts.

2. Nested Loops Review

Nested loops are loops placed inside other loops. The inner loop completes all its iterations for each single iteration of the outer loop. Nested loops are essential for working with multi-dimensional data and generating patterns.

```
for i in range(1, 4):
    for j in range(2, 4):
        print(i, "and", j)
```

Output:
1 and 2
1 and 3
2 and 2
2 and 3
3 and 2
3 and 3

3. Conditional Statements Review

if-elif-else statements allow programs to make decisions based on conditions. Multiple conditions can be chained using elif (else if) to handle various scenarios.

```
if condition1:
    # code
elif condition2:
    # code
else:
    # code
```

4. String Multiplication for Patterns

In Python, strings can be multiplied by integers to repeat them. This is useful for generating patterns efficiently without nested loops:

```
print("*" * 5)  # Output: *****
```

5. Dictionary Iteration Review

Dictionaries store key-value pairs. Iterating over a dictionary using the keys() method or directly over the dictionary allows access to each key and its corresponding value:

```
for key in dictionary:
    print(key, dictionary[key])
```

6. Input Validation

Input validation ensures that user-provided data meets expected criteria before processing. This prevents errors and ensures program reliability:

```
if tot <= 0 or got > tot:
    print("Invalid input!")
```

7. Grade Calculation Logic

Grade calculation involves computing a percentage and mapping it to letter grades using conditional statements. Each grade range corresponds to a specific level of achievement.

8. Combining Concepts

Real-world programming problems often require combining multiple concepts: loops for iteration, conditionals for decision-making, functions for modularity, and data structures for organization.

Lab Work:

Task 1: Number Pairs Using Nested Loops

Write a program that prints all pairs of numbers using nested for loops.

Code Summary:

This task demonstrates nested for loops where the outer loop controls the first number and the inner loop controls the second number, generating all possible combinations.

Key Code Lines:

```
# Number Pairs Using Nested Loops
for i in range(1, 4):
    for j in range(2, 4):
        print(i, "and", j)
```

Output:

Task 2: Student Grade Calculator Using For Loop

Write a program that calculates grades for multiple students using a for loop and conditional statements.

Code Summary:

This program uses a for loop to process multiple students. For each student, it calculates the percentage and assigns a grade based on conditional checks. Input validation ensures data integrity.

Key Code Lines:

```
# Student Grade Calculator (For Loop)
num = int(input("Enter total number of students: "))
for i in range(num):
    nm = input("Enter student name: ")
    rn = int(input("Enter roll number: "))
    got = int(input("Enter obtained marks: "))
    tot = int(input("Enter total marks: "))
    if tot <= 0 or tot > 300 or got > tot:
```

```
        print("Enter valid numbers")
    else:
        perc = (got / tot) * 100
        print("Percentage:", perc)
        if perc >= 90: print("Grade: A+")
        elif perc >= 85: print("Grade: A-")
        elif perc >= 80: print("Grade: B+")
        elif perc >= 75: print("Grade: B-")
        elif perc >= 70: print("Grade: C+")
        elif perc >= 65: print("Grade: C-")
        else: print("Grade: F")
```

Output:

Task 3: Items Dictionary Display

Create a dictionary of items and display all key-value pairs using iteration.

Code Summary:

This task demonstrates dictionary creation and iteration. The for loop iterates over dictionary keys, and each key is used to access its corresponding value.

Key Code Lines:

```
# Items Dictionary Display
items = {"Mango": "a fruit", "Orange": "a fruit", "Grapes": "a fruit"}
for k in items:
    print(k, ":", items[k])
```

Output:

Task 4: Right Triangle Star Pattern

Write a program that prints a right triangle star pattern using string multiplication.

Code Summary:

This task uses string multiplication to generate a right triangle pattern efficiently. The number of stars in each row equals the row number.

Key Code Lines:

```
# Right Triangle Star Pattern
r = 7
for i in range(1, r + 1):
    print("*" * i)
```

Output:

Task 5: Student Grade Calculator Using While Loop

Write a program that calculates grades using a while loop instead of a for loop.

Code Summary:

This task demonstrates the same grade calculation logic but using a while loop with a manual counter, showing the flexibility of loop choice based on requirements.

Key Code Lines:

```
# Student Grade Calculator (While Loop)
num = int(input("Enter total number of students: "))
i = 1
while i <= num:
    nm = input("Enter student name: ")
    rn = int(input("Enter roll number: "))
    got = int(input("Enter obtained marks: "))
    tot = int(input("Enter total marks: "))
    if tot <= 0 or tot > 300 or got > tot:
        print("Enter valid numbers")
    else:
        perc = (got / tot) * 100
        print("Percentage:", perc)
        if perc >= 90: print("Grade: A+")
        elif perc >= 85: print("Grade: A-")
        elif perc >= 80: print("Grade: B+")
        elif perc >= 75: print("Grade: B-")
        elif perc >= 70: print("Grade: C+")
        elif perc >= 65: print("Grade: C-")
        else: print("Grade: F")
    i += 1
```

Output:

Task 6: Pyramid Star Pattern

Write a program that prints a pyramid star pattern with proper spacing.

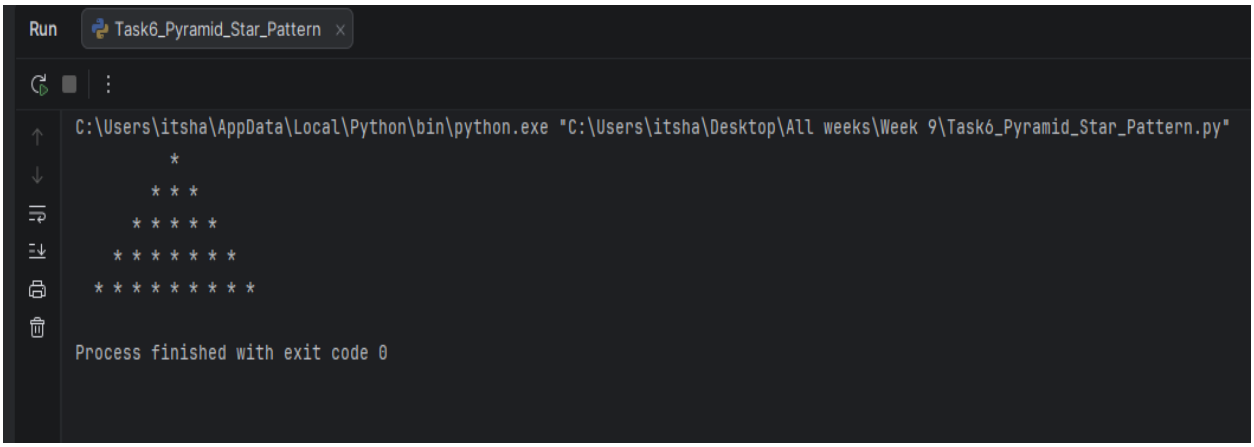
Code Summary:

This task creates a pyramid pattern by printing spaces before stars. The number of spaces decreases while the number of stars increases as we move down the rows.

Key Code Lines:

```
# Pyramid Star Pattern
r = 6
for i in range(1, r):
    print("  " * (r - i), end="")
    print("* " * (2*i - 1))
```

Output:



Conclusions:

• Successfully reviewed and applied fundamental programming concepts from Weeks 1-8

• Demonstrated proficiency in both for and while loops for iterative problem solving

• Applied nested loops for generating complex patterns and combinations

• Combined conditional statements with loops for decision-making in iterative processes

• Worked with dictionaries for storing and displaying structured data

• Implemented input validation to ensure data integrity

• Gained confidence in solving comprehensive programming problems combining multiple concepts

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 10	**CLOs to be covered:** CLO2
Lab Title: Recursion	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO2	Design and implement modular programs using functions, recursion, and reusable code constructs.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**

Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 10

Lab Goals / Objectives:

• Understand the concept of recursion in Python

• Learn the components of a recursive function

• Differentiate between iterative and recursive approaches

• Implement recursive solutions for mathematical problems

• Understand the call stack and recursion depth

• Learn about base cases and recursive cases

• Apply recursion to solve the Fibonacci sequence problem

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. What is Recursion?

Recursion is a programming technique where a function calls itself to solve a problem by breaking it down into smaller, similar subproblems. A recursive function solves a problem by solving smaller instances of the same problem.

2. Components of a Recursive Function

Every recursive function must have two essential components:

• Base Case: The condition that stops the recursion. It provides a direct answer without making further recursive calls.

• Recursive Case: The part where the function calls itself with a modified (usually smaller) version of the original problem.

3. How Recursion Works

When a recursive function is called, each call is placed on the call stack. The function keeps calling itself until the base case is reached. Then, the results are combined as the stack unwinds back to the original call.

4. Recursion vs Iteration

Recursion and iteration (loops) can often solve the same problems. Recursion provides elegant solutions for problems with recursive structure (like trees), while iteration is generally more memory-efficient.

| Aspect | Recursion | Iteration |

| Code | Often simpler | Can be more verbose |

| Memory | Uses call stack | Uses less memory |

| Speed | Can be slower | Generally faster |

| Risk | Stack overflow | Infinite loops |

5. Fibonacci Sequence

The Fibonacci sequence is a series where each number is the sum of the two preceding ones: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Mathematically: $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$ for $n > 1$

This is naturally expressed recursively since each Fibonacci number depends on the two previous ones.

6. Built-in Functions Review

Python provides many useful built-in functions that complement recursive programming:

• len() - Returns the length of a sequence

• max() - Returns the maximum of given values

• min() - Returns the minimum of given values

• type() - Returns the type of an object

• range() - Generates a sequence of numbers

7. Nested Loops for Tables

Nested loops are essential for generating tables and matrices. The outer loop controls rows and the inner loop controls columns:

```
for i in range(1, 11):
    for j in range(1, 11):
        print(i * j, end="\t")
    print()
```

8. String Alignment with print()

The print() function's end parameter can be used to control spacing. Using \t (tab) as the separator aligns output in columns:

```
print(value, end="\t")  # Prints with tab instead of newline
```

9. Recursion Depth Limit

Python has a default recursion depth limit (usually 1000) to prevent stack overflow. This can be checked and modified using sys.getrecursionlimit() and sys.setrecursionlimit().

10. When to Use Recursion

Recursion is ideal for problems with:

• Recursive mathematical definitions (factorial, Fibonacci)

• Tree and graph traversals

• Divide-and-conquer algorithms

• Problems that can be broken into similar subproblems

Lab Work:

Task 1: Fibonacci Number Using Recursion

Write a recursive function to calculate the Fibonacci number at a given position.

Code Summary:

This is the classic recursive Fibonacci implementation. The base cases handle positions 0 and 1, while the recursive case sums the two preceding Fibonacci numbers.

Key Code Lines:

```
# Fibonacci Using Recursion
def fib(n):
    if n == 0 or n == 1:
        return n
    return fib(n-1) + fib(n-2)

num = int(input("Enter a number: "))
print("Fibonacci at position", num, "is:", fib(num))
```

Output:

Task 2: Factorial Using Recursion

Write a recursive function to calculate the factorial of a number.

Code Summary:

Factorial is a natural fit for recursion: n! = n * (n-1)!. The base case is 0! = 1, and the recursive case multiplies n by the factorial of n-1.

Key Code Lines:

```
# Factorial Using Recursion
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

num = int(input("Enter a number: "))
print("Factorial of", num, "is:", factorial(num))
```

Output:

Task 3: Countdown Using Recursion

Write a recursive function that prints numbers counting down from a given value to 1.

Code Summary:

This task demonstrates recursion without accumulating a return value: the function prints the current number, then calls itself with a decremented value until the base case (0) is reached.

Key Code Lines:

```
# Task 3: Countdown Using Recursion
def show(n):
    if n == 0:
        return
    print(n)
    show(n - 1)

show(5)
```

Output:

Conclusions:

• Successfully understood the concept and mechanics of recursion

• Learned the two essential components: base case and recursive case

• Differentiated between recursive and iterative approaches

• Implemented recursive solutions for Fibonacci and factorial problems

• Understood how the call stack works during recursive execution

• Reviewed and applied built-in functions alongside user-defined functions

• Applied nested loops for generating formatted tables and patterns

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 11	**CLOs to be covered:** CLO4
Lab Title: Exception Handling	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO4	Implement error handling and debugging techniques to build robust, fault-tolerant programs.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 11

Lab Goals / Objectives:

• Understand what exceptions are and why they occur

• Learn to use try-except blocks for error handling

• Handle multiple exception types

• Use the else and finally clauses

• Raise exceptions intentionally with the raise keyword

• Understand common built-in exception types

• Write robust programs that handle errors gracefully

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. What are Exceptions?

An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions. When an error occurs, Python raises an exception which can be caught and handled to prevent the program from crashing.

2. Common Types of Exceptions

Python has many built-in exception types:

• ZeroDivisionError: Raised when dividing by zero

• ValueError: Raised when an operation receives an argument with the right type but inappropriate value

• TypeError: Raised when an operation is applied to an object of inappropriate type

• IndexError: Raised when a sequence index is out of range

• KeyError: Raised when a dictionary key is not found

• FileNotFoundError: Raised when a file operation fails

• NameError: Raised when a variable is not defined

3. The try-except Block

The try-except block is used to catch and handle exceptions. Code that might raise an exception is placed in the try block, and the handling code is placed in the except block:

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print('Cannot divide by zero!')
```

4. Catching Multiple Exceptions

Multiple except blocks can be used to handle different types of exceptions. A single except can also catch multiple exceptions using a tuple:

```
try:
    x = int(input('Enter a number: '))
    result = 10 / x
except ValueError:
    print('Invalid input!')
except ZeroDivisionError:
    print('Cannot divide by zero!')
```

5. The else Clause

The else clause executes if no exception was raised in the try block. It is useful for code that should run only when the try block succeeds:

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print('Error!')
else:
    print('Result:', result)
```

6. The finally Clause

The finally clause always executes, regardless of whether an exception occurred. It is commonly used for cleanup operations like closing files:

```
try:
    f = open('file.txt')
except FileNotFoundError:
    print('File not found!')
finally:
    print('Cleanup complete.')
```

7. The raise Keyword

The raise keyword allows you to intentionally raise an exception. This is useful for enforcing constraints or signaling errors in your own code:

```
def check_age(age):
    if age < 0:
        raise ValueError('Age cannot be negative!')
```

8. Exception Hierarchy

All built-in exceptions in Python inherit from the BaseException class. The most common base class for user-defined exceptions is Exception. Catching Exception will catch most common errors.

9. Best Practices

• Catch specific exceptions rather than the general Exception class

• Use finally for cleanup operations that must always run

• Do not use exceptions for normal flow control

• Provide meaningful error messages to help debugging

• Log exceptions for troubleshooting in production code

10. Sets Review

Sets are unordered collections of unique elements. They support mathematical operations like union, intersection, and difference. The issubset() and issuperset() methods check set relationships.

Lab Work:

Task 1: Basic try-except

Write a program that takes a number as input inside a try block and catches any input error with a general except.

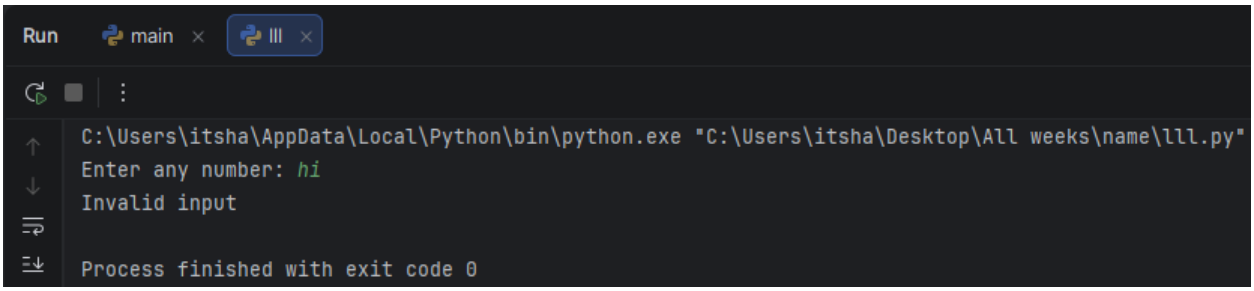
Code Summary:

This task shows the simplest form of exception handling: risky code (converting input to an integer) is placed in a try block, and a bare except catches any error that occurs, preventing a crash.

Key Code Lines:

```
# Task 1: Basic try-except
try:
    number = int(input("Enter any number: "))
    print("Hello World")
except:
    print("Invalid input")
```

Output:



Task 2: Catching the Exception Object

Write a program that catches an input error and prints the actual exception message using 'except Exception as e'.

Code Summary:

This task demonstrates capturing the exception object itself with 'as e', which lets the program display Python's own error description instead of a generic message.

Key Code Lines:

```
# Task 2: Catching the Exception Object
try:
    number = int(input("Enter any number: "))
    print("Hello World")
except Exception as e:
    print(e)
```

Output:

Task 3: Division by Zero Handling

CODE:

```
num = int(input("Enter a number: "))
try:
    ans = num / 0
    print("Result:", ans)
except ZeroDivisionError as e:
    print("Cannot divide by zero!")
    print("Error:", e)
```

Output:

Conclusions:

- Successfully understood what exceptions are and why they occur
- Learned to use try-except blocks for graceful error handling
- Implemented handling for multiple exception types
- Understood the purpose and use of else and finally clauses
- Learned to raise exceptions intentionally with the raise keyword
- Reviewed set operations including subset and superset checking
- Wrote robust programs that handle errors without crashing

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 12	**CLOs to be covered:** CLO2
Lab Title: Lambda Functions	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO2	Design and implement modular programs using functions, recursion, and reusable code constructs.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 12

Lab Goals / Objectives:

• Understand the concept of lambda (anonymous) functions

• Learn the syntax and structure of lambda functions

• Differentiate between lambda functions and regular functions

• Use lambda functions with built-in functions like map(), filter(), and sorted()

• Apply lambda functions to practical programming problems

• Combine lambda with conditional expressions

• Understand when to use lambda vs regular functions

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. What are Lambda Functions?

Lambda functions (also called anonymous functions) are small, restricted functions that are not bound to an identifier. They are defined using the lambda keyword instead of def. Lambda functions can have any number of arguments but only one expression.

2. Lambda Syntax

The syntax of a lambda function is:

```
lambda arguments : expression
```

The expression is evaluated and returned when the lambda is called. Lambda functions can be assigned to variables or used inline.

3. Lambda vs Regular Functions

| Feature | Lambda | Regular Function |

| Syntax | lambda args: expr | def name(args): return expr |

| Name | Anonymous (no name) | Has a name |

| Body | Single expression | Multiple statements |

| Return | Implicit return | Explicit return statement |

| Use | Short, simple operations | Complex logic |

4. Lambda with Multiple Arguments

Lambda functions can accept multiple arguments separated by commas:

```
# Lambda with two arguments
multiply = lambda x, y: x * y
print(multiply(5, 3))  # Output: 15
```

5. Lambda with Conditional Expressions

Lambda functions can include conditional expressions for simple decision-making:

```
# Lambda to find maximum
maximum = lambda a, b: a if a > b else b
print(maximum(10, 20))  # Output: 20
```

6. The map() Function

The map() function applies a given function to every item of an iterable and returns an iterator. It is commonly used with lambda:

```
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x**2, numbers))
print(squares)  # [1, 4, 9, 16, 25]
```

7. The filter() Function

The filter() function constructs an iterator from elements of an iterable for which a function returns True:

```
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)  # [2, 4, 6]
```

8. Using Lambda with sorted()

The sorted() function accepts a key parameter that can be a lambda function for custom sorting:

```
students = [('Ali', 85), ('Ahmed', 92), ('Bilal', 78)]
sorted_by_score = sorted(students, key=lambda x: x[1])
```

9. When to Use Lambda Functions

• For short, throwaway functions that are used once

• As arguments to higher-order functions (map, filter, sorted)

• For simple transformations that fit in one expression

• When defining a full function would be unnecessarily verbose

10. When NOT to Use Lambda Functions

• For complex logic requiring multiple statements

• When the function needs to be reused extensively

• When readability would be compromised

• For functions that need documentation (docstrings)

Lab Work:

Task 1: Greet Function with Name and Age

Write a program that defines a function to greet a person by name and display their age.

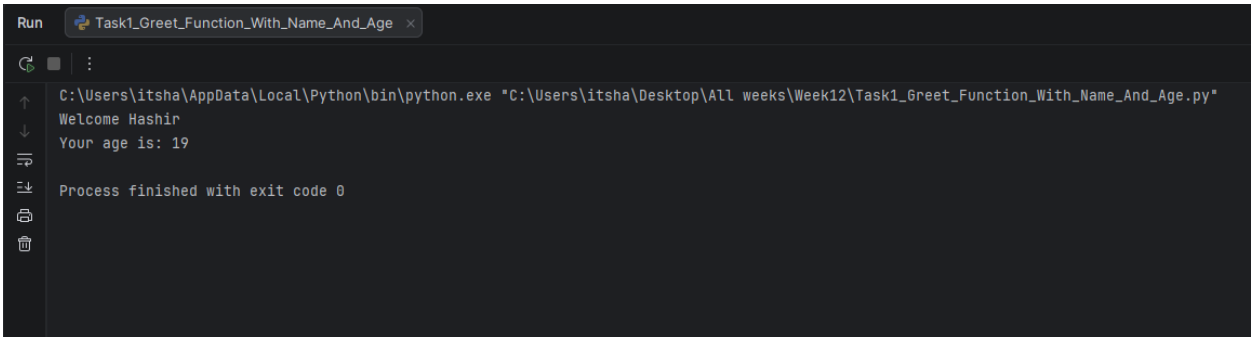
Code Summary:

This task reviews regular function definition with multiple parameters as a foundation for comparing with lambda functions.

Key Code Lines:

```
# Greet Function with Name and Age
def greet(name, age):
    print("Welcome", name)
    print("Your age is:", age)

greet("Hashir", 19)
```

Output:



Task 2: Lambda Square Function

Write a lambda function that calculates the square of a number.

Code Summary:

This task demonstrates the basic lambda syntax. The lambda takes one argument and returns its square, achieving in one line what would normally require a full function definition.

Key Code Lines:

```
# Lambda Square Function
square = lambda number: number ** 2
print(square(2))

# Lambda with two arguments
multiply = lambda x, y: x * y
print(multiply(5, 5))

# Lambda with three arguments
product = lambda x, y, z: x * y * z
print(product(5, 5, 5))
```

Output:

Task 3: Lambda with map() Function

Use lambda with the map() function to square all elements in a list.

Code Summary:

This task demonstrates combining lambda with map() to apply an operation to every element of a list efficiently without writing an explicit loop.

Key Code Lines:

```
# Lambda with map()
x = [1, 2, 3, 4, 5]
```

```
squared = list(map(lambda x: x * x, x))
print(squared)
```

Output:

Task 4: Lambda Maximum with Conditional

Write a lambda function that returns the maximum of two numbers using a conditional expression.

Code Summary:

This task demonstrates using a conditional expression within a lambda to make decisions. The syntax 'a if condition else b' evaluates to a if the condition is True, otherwise b.

Key Code Lines:

```
# Lambda Maximum with Conditional
maximum = lambda a, b: a if a > b else b
print("Max of 10, 20:", maximum(10, 20))
print("Max of 50, 30:", maximum(50, 30))
```

Output:

Task 5: Lambda with filter() Function

Use lambda with the filter() function to extract even numbers from a list.

Code Summary:

This task shows how lambda combined with filter() creates a concise way to select elements from a list that satisfy a condition.

Key Code Lines:

```
# Lambda with filter()
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print("Even numbers:", evens)
```

Output:

Conclusions:

• Successfully understood the concept and syntax of lambda (anonymous) functions

• Differentiated between lambda functions and regular named functions

• Learned to use lambda with multiple arguments and conditional expressions

• Applied lambda with map() to transform all elements of a list

• Applied lambda with filter() to select elements based on a condition

• Understood the appropriate use cases for lambda functions

• Gained the ability to write more concise and functional Python code

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 13	**CLOs to be covered:** CLO2
Lab Title: MAP, Filter and Reduce	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO2	Design and implement modular programs using functions, recursion, and reusable code constructs.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**

Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 13

Lab Goals / Objectives:

• Understand functional programming concepts in Python

• Master the map() function for transforming iterables

• Master the filter() function for selecting elements

• Learn the reduce() function from the functools module

• Combine lambda functions with map, filter, and reduce

• Differentiate between map/filter/reduce and list comprehensions

• Apply functional programming techniques to practical problems

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. Functional Programming in Python

Functional programming is a programming paradigm where computation is treated as the evaluation of mathematical functions. Python supports functional programming through tools like map(), filter(), reduce(), and lambda functions. These tools allow writing cleaner, more declarative code.

2. The map() Function

The map() function applies a specified function to every item of an iterable (like a list) and returns an iterator with the results. It is one of the most commonly used functional programming tools in Python.

Syntax: map(function, iterable)

```
numbers = [1, 2, 3, 4]
squares = list(map(lambda x: x**2, numbers))
# Result: [1, 4, 9, 16]
```

The map() function is more concise than writing an explicit for loop and is particularly useful when the transformation logic is simple.

3. The filter() Function

The filter() function constructs an iterator from elements of an iterable for which a function returns True. It is used to select elements that satisfy a specific condition.

Syntax: filter(function, iterable)

```
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
# Result: [2, 4, 6]
```

The function passed to filter() should return a boolean value. Only elements for which the function returns True are included in the result.

4. The reduce() Function

The reduce() function from the functools module applies a rolling computation to sequential pairs of values in an iterable. It reduces the iterable to a single value.

Syntax: reduce(function, iterable, initializer)

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)
# Result: 120 (1*2*3*4*5)
```

reduce() is useful for aggregations like summing all elements, finding the product, or finding the maximum/minimum.

5. functools Module

The functools module provides higher-order functions that act on or return other functions. Key functions include:

• reduce() - Rolling computation on a list

• partial() - Creates a partial function with fixed arguments

• wraps() - Decorator for preserving function metadata

6. map() vs List Comprehension

Both map() and list comprehensions can transform iterables. List comprehensions are generally preferred in Python for readability:

```
# Using map
squares = list(map(lambda x: x**2, numbers))

# Using list comprehension (preferred)
squares = [x**2 for x in numbers]
```

However, map() can be more efficient for large datasets and when the transformation function is already defined.

7. Combining map() and filter()

map() and filter() can be combined for powerful data processing pipelines:

```
numbers = [1, 2, 3, 4, 5, 6]
# Square only even numbers
result = list(map(lambda x: x**2, filter(lambda x: x % 2 == 0, numbers)))
# Result: [4, 16, 36]
```

8. Practical Applications

• map(): Data transformation, type conversion, normalization

• filter(): Data cleaning, selecting valid records, removing outliers

• reduce(): Aggregations, cumulative calculations, finding extremes

Lab Work:

Task 1: Square Function

Write a regular function to calculate the square of a number.

Code Summary:

This task establishes the foundation by defining a regular function that will be compared with lambda and map() approaches.

Key Code Lines:

```
# Square Function
def square(number):
    return number ** 2
print(square(2))
```

Output:

Task 2: Lambda Square and Multiplication

Write lambda functions for squaring and multiplication operations.

Code Summary:

This task demonstrates using lambda for simple mathematical operations, showing the conciseness of lambda syntax compared to regular functions.

Key Code Lines:

```
# Lambda Square and Multiplication
a = lambda x: x * x
print(a(5))

a = lambda x, y: x * y
print(a(5, 5))

a = lambda x, y, z: x * y * z
print(a(5, 5, 5))
```

Output:

Task 3: map() with Lambda

Use the map() function with a lambda to square all elements in a list.

Code Summary:

This task demonstrates the functional programming approach: applying a transformation to every element of a collection without explicit loops.

Key Code Lines:

```
# map() with Lambda
x = [1, 2, 3, 4, 5]
```

```
a = list(map(lambda x: x * x, x))
print(a)
```

Output:

Code Summary:

This task shows how filter() selects elements from a list based on a condition, returning only those elements that satisfy the predicate.

Key Code Lines:

```
# filter() with Lambda
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)
```

Output:

Task 5: reduce() from functools

Use the reduce() function to calculate the product of all numbers in a list.

Code Summary:

This task demonstrates reduce() for aggregation. The lambda takes two arguments and applies the operation cumulatively from left to right.

Key Code Lines:

```
# reduce() from functools
from functools import reduce
numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)
print("Product:", product)

# Finding maximum using reduce
maximum = reduce(lambda x, y: x if x > y else y, numbers)
print("Maximum:", maximum)
```

Output:

Conclusions:

• Successfully understood functional programming concepts in Python

• Mastered the map() function for transforming all elements of an iterable

• Mastered the filter() function for selecting elements based on conditions

• Learned the reduce() function for aggregating iterables to a single value

• Combined lambda functions with map(), filter(), and reduce() for concise code

• Understood the differences and trade-offs between functional and imperative approaches

• Applied functional programming techniques to solve practical problems

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 14	**CLOs to be covered:** CLO2
Lab Title: OS Library, if __name__ == "__main__", is vs ==	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO2	Design and implement modular programs using functions, recursion, and reusable code constructs.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**

Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 14

Lab Goals / Objectives:

• Understand the OS module and its functions for system operations

• Learn file and directory management using the os module

• Understand the purpose of if __name__ == '__main__' construct

• Differentiate between the 'is' operator and '==' operator

• Learn to create and manage modules in Python

• Understand Python program execution flow

• Apply OS operations for practical file management tasks

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. The OS Module

The os module in Python provides functions for interacting with the operating system. It allows you to perform file and directory operations, manage processes, and access environment variables. The os module is part of Python's standard library and does not require installation.

2. Important OS Module Functions

• os.getcwd() - Returns the current working directory as a string

• os.chdir(path) - Changes the current working directory to the specified path

• os.listdir(path) - Returns a list of all files and directories in the specified path

• os.mkdir(path) - Creates a new directory at the specified path

• os.makedirs(path) - Creates a directory and all intermediate-level directories

• os.rmdir(path) - Removes an empty directory

• os.remove(path) - Deletes a file at the specified path

• os.rename(src, dst) - Renames a file or directory from src to dst

• os.path.exists(path) - Returns True if the path exists

• os.path.isfile(path) - Returns True if the path is a file

• os.path.isdir(path) - Returns True if the path is a directory

3. Working with Directories

The os module provides comprehensive directory management capabilities. You can create, navigate, list, and remove directories programmatically:

```
import os
print(os.getcwd())  # Current directory
os.mkdir('newfolder')  # Create directory
print(os.listdir('.'))  # List contents
```

4. if __name__ == '__main__'

This construct is a fundamental Python idiom. When a Python file is run directly, the special variable __name__ is set to '__main__'. However, when the file is imported as a module, __name__ is set to the module's name.

```
def main():
    print('Running directly')

if __name__ == '__main__':
    main()
```

This allows code to behave differently when run as a script versus when imported as a module. It is the standard way to write reusable Python modules.

5. Why Use if __name__ == '__main__'?

• Prevents code from running when the module is imported

• Allows the file to be both a reusable module and a standalone program

• Makes the code more organized and testable

• Follows Python best practices for module design

6. The 'is' Operator vs '==' Operator

The == operator compares the values of two objects for equality.

The is operator checks whether two references point to the same object in memory (identity comparison).

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)  # True (same values)
print(a is b)  # False (different objects)
```

7. When to Use 'is'

The is operator is primarily used for comparing with singleton objects like None, True, and False:

```
if x is None:
    print('x is None')
```

Using is for value comparison can lead to unexpected behavior because it checks identity, not equality.

8. Python Modules and Imports

A module is a Python file containing functions, classes, and variables that can be imported into other programs. The import statement makes a module's contents available:

```
import os
import Hashir  # Custom module
Hashir.welcome()
```

9. String Methods: isdigit(), isspace()

• str.isdigit() - Returns True if all characters in the string are digits

• str.isspace() - Returns True if all characters are whitespace characters

These methods are useful for input validation and data cleaning.

10. The globals() Function Revisited

The globals() function returns a dictionary of the current global symbol table. It can be used to access global variables by their string names, which is useful for dynamic variable access.

Lab Work:

Task 1: Creating a Python Module

Create a custom Python module with functions and demonstrate importing it.

Code Summary:

This task demonstrates creating a reusable Python module. The module defines two functions (welcome and Goodbye) and uses if __name__ == '__main__' to run test code only when the module is executed directly.

Key Code Lines:

```
# Module: Hashir.py
def welcome():
    print("Welcome Message")

def Goodbye():
    print("Goodbye Message")

if __name__ == '__main__':
    welcome()
    Goodbye()
```

Output:

Task 2: Importing a Custom Module

Write a program that imports and uses functions from a custom module.

Code Summary:

This task shows how to import a custom module and call its functions. The module's functions are accessed using the module name as a prefix.

Key Code Lines:

```
# Importing Custom Module
import Hashir
Hashir.welcome()
Hashir.Goodbye()
```

Output: No real Ouput required for this task according to sir

Task 3: OS Commands Demonstration

Write a program that demonstrates five common OS module functions.

Code Summary:

This task demonstrates practical OS operations: getting the current directory, listing files, renaming files, deleting files, and removing directories.

Key Code Lines:

```
# Five OS Python Commands
import os

os.getcwd()           # Get current working directory
os.listdir()          # List all files in current directory
os.rename("old.py", "new.py")  # Rename a file
os.remove("file.py")  # Delete a file
os.rmdir("folder")    # Remove directory
```

Output: No real Ouput required for this task according to sir

Task 4: Creating Multiple Directories

Write a program that creates multiple directories using a loop.

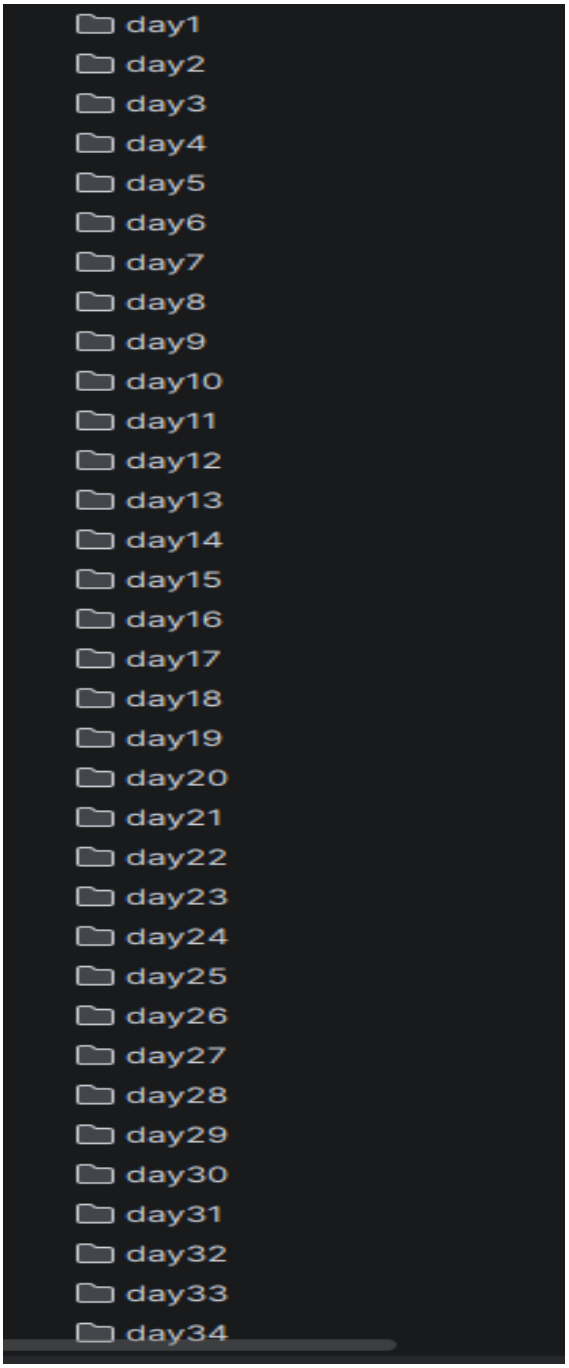
Code Summary:

This task demonstrates creating directories programmatically using os.mkdir() inside a loop, which is useful for batch directory creation.

Key Code Lines:

```
# Creating Multiple Directories
import os
os.mkdir("name")
for i in range(0, 100):
    os.mkdir(f"day{i+1}")
```

Output:



Task 5: is vs == Demonstration

Write a program that demonstrates the difference between 'is' and '==' operators.

Code Summary:

This task clearly demonstrates that == compares values while is compares object identity (memory location). Two lists with the same values are equal but not identical.

Key Code Lines:

```
# is vs == Demonstration
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print("a == b:", a == b)   # True (same values)
print("a is b:", a is b)   # False (different objects)
print("a is c:", a is c)   # True (same object)
print("id(a):", id(a))
print("id(b):", id(b))
print("id(c):", id(c))
```

Output:

Conclusions:

• Successfully understood and applied the OS module for system operations

• Learned to create, import, and use custom Python modules

• Understood the purpose and importance of if __name__ == '__main__'

• Differentiated between the 'is' operator (identity) and '==' operator (equality)

• Applied OS functions for file and directory management

• Understood Python program execution flow and module design

• Gained practical skills in writing reusable and well-structured Python code

University Of Engineering and Technology, Lahore

Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 15	**CLOs to be covered:** CLO3
Lab Title: GUI using Qt Designer	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	**Section:** CYS-A

Lab Evaluation:

CLO3	Use appropriate data structures to organize, store, and manipulate data efficiently.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						**/5**

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric

Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 15

Lab Goals / Objectives

- Understand the concept of Graphical User Interface (GUI) programming
- Learn about PyQt5 and its core components
- Understand Qt Designer for visual GUI design
- Learn about widgets, layouts, and signals/slots
- Convert .ui files to Python code using pyuic5
- Understand event-driven programming paradigm
- Create a functional GUI application

Equipment Required

- Computer System with Python 3.x installed
- PyQt5 library installed (pip install PyQt5)
- Qt Designer application
- Text Editor or IDE (PyCharm, VS Code, or IDLE)
- Operating System: Windows / Linux / macOS

Theory

1. What is GUI Programming?

Graphical User Interface (GUI) programming involves creating visual interfaces for applications that users can interact with using graphical elements like buttons, text fields, labels, and menus. Unlike command-line interfaces, GUIs provide a more intuitive and user-friendly experience.

2. Introduction to PyQt5

PyQt5 is a set of Python bindings for the Qt5 application framework. Qt is a comprehensive C++ library for developing cross-platform applications with GUIs. PyQt5 allows Python programmers to create complex GUI applications with ease.

3. Core PyQt5 Modules

- QtCore: Core non-GUI functionality (event loop, timers, data structures)
- QtGui: GUI components (fonts, colors, icons, images)
- QtWidgets: UI elements (buttons, labels, text fields, layouts)

4. Common PyQt5 Widgets

- QApplication, QWidget, QLabel, QPushButton, QLineEdit, QTextEdit
- QComboBox, QCheckBox, QRadioButton, QSlider, QProgressBar
- QMessageBox, QMainWindow, QTableWidget, QDialog, QTabWidget

5. Qt Designer

Qt Designer is a visual tool for designing GUIs. It allows dragging and dropping widgets onto a canvas, setting properties, and defining layouts. The designed interface is saved as a .ui file which can be converted to Python code.

9. The CoreStock Application

CoreStock is the GUI application built for this lab using PyQt5 and SQLite. It demonstrates practical application of GUI programming concepts including form design, data entry, table views, and interactive controls across a complete, multi-screen desktop workflow.

Project Proposal

Introduction

CoreStock is a desktop inventory and sales-floor management application built for a small-to-medium PC hardware retail store. It replaces manual, paper-based or spreadsheet-based stock tracking with a single, role-based desktop application that lets store staff manage products, record stock movements, maintain a supplier list, and generate stock reports — all from one consistent graphical interface.

The project was developed entirely in Python using the PyQt5 framework for the graphical interface and SQLite as the embedded database engine, and is version-controlled on GitHub.

Objectives

- Provide a secure, login-protected entry point so only authorized staff can access store data.
- Maintain a complete, searchable product catalogue with category, price, quantity, and a configurable low-stock alert threshold.
- Allow staff to record every stock movement and have product quantities update automatically as a result.

- Maintain a basic supplier directory for procurement contacts.
- Summarize stock activity in a reporting view, exportable as CSV.
- Present all of the above through a clean, modern, consistent PyQt5 interface with a custom stylesheet.

Scope

CoreStock is scoped as a single-store, single-machine desktop application rather than a networked, multi-branch system — there is no cloud sync, multi-user concurrency, or external API integration. Within this scope it fully supports user authentication, product inventory management, stock-in/stock-out tracking with automatic quantity adjustment, supplier record-keeping, and exportable stock reports.

Tools and Technologies Used

- Programming Language: Python 3
- GUI Framework: PyQt5 (QtWidgets, QtCore, QtGui)
- Database Engine: SQLite 3 (via Python's built-in sqlite3 module)
- IDE: PyCharm
- Version Control: Git and GitHub
- Architecture Style: Modular, single-window desktop app with tabbed navigation

Project Implementation — Source Code Walkthrough

The CoreStock PC Store Management System is organized into eight Python modules under a single package. Each module below is presented with a short description of its role followed by its complete source code.

main.py

Application entry point. Initializes the SQLite database, creates the QApplication instance, applies a global stylesheet (Segoe UI font, indigo accent color, rounded buttons and inputs), and launches the Login window.

```
import sys

from PyQt5.QtWidgets import QApplication

import database

from login import LoginWindow



database.setup()

app = QApplication(sys.argv)

app.setStyleSheet("""

    QWidget { font-family: Segoe UI; font-size: 13px; color: #1c1c1e; background: white; }

    QPushButton { background: #6366f1; color: white; border-radius: 7px; padding: 6px 14px; font-
weight: bold; border: none; }

    QPushButton:hover { background: #4f46e5; }

    QLineEdit, QComboBox, QSpinBox, QDoubleSpinBox { border: 1px solid #e8e8ed; border-radius: 6px;
padding: 5px 8px; background: #f8f8fa; }

    QLineEdit:focus { border: 1px solid #6366f1; background: white; }

    QTableWidget { background: white; border: 1px solid #e8e8ed; gridline-color: #f3f3f6; }

    QTableWidget::item:selected { background: #eef2ff; color: #4338ca; }

    QHeaderView::section { background: #f8f8fa; color: #6b6b6b; font-weight: bold; padding: 7px;
border: none; border-bottom: 1px solid #e8e8ed; }

    QTabBar::tab { background: transparent; color: #a1a1aa; padding: 9px 20px; border-bottom: 2px
solid transparent; }

    QTabBar::tab:selected { color: #6366f1; border-bottom: 2px solid #6366f1; font-weight: bold; }

    QTabWidget::pane { border: 1px solid #e8e8ed; }
```

```
    QDialog { background: white; }
""")
window = LoginWindow()
window.show()
sys.exit(app.exec_())
```

database.py

Data access layer. Every database operation in the app goes through this module — creating the four tables (users, products, suppliers, movements) on first run, seeding two default accounts, and providing functions for login checks, product/supplier CRUD, low-stock lookups, and stock movement recording.

```
import sqlite3



def setup():
    con = sqlite3.connect("store.db")
    cur = con.cursor()
    cur.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, username TEXT, password
TEXT, role TEXT)")
    cur.execute("CREATE TABLE IF NOT EXISTS products (id INTEGER PRIMARY KEY, name TEXT, category
TEXT, price REAL, quantity INTEGER, low_limit INTEGER)")
    cur.execute("CREATE TABLE IF NOT EXISTS suppliers (id INTEGER PRIMARY KEY, name TEXT, contact
TEXT, email TEXT)")
    cur.execute("CREATE TABLE IF NOT EXISTS movements (id INTEGER PRIMARY KEY, product TEXT, type
TEXT, qty INTEGER, date TEXT)")
    cur.execute("SELECT * FROM users WHERE username='admin'")
    if not cur.fetchone():
        cur.execute("INSERT INTO users VALUES (NULL,'admin','admin123','Admin')")
        cur.execute("INSERT INTO users VALUES (NULL,'staff','staff123','Staff')")
    con.commit()
    con.close()
```

```
def check_login(u, p):
    con = sqlite3.connect("store.db")
    cur = con.cursor()
    cur.execute("SELECT * FROM users WHERE username=? AND password=?", (u, p))
    row = cur.fetchone()
    con.close()
    return row

def get_products():
    con = sqlite3.connect("store.db")
    cur = con.cursor()
    cur.execute("SELECT * FROM products")
    rows = cur.fetchall()
    con.close()
    return rows

def add_product(name, cat, price, qty, low):
    con = sqlite3.connect("store.db")
    cur = con.cursor()
    cur.execute("INSERT INTO products VALUES (NULL,?,?,?,?,?)", (name, cat, price, qty, low))
    con.commit()
    con.close()

def update_product(pid, name, cat, price, qty, low):
    con = sqlite3.connect("store.db")
    cur = con.cursor()
    cur.execute("UPDATE products SET name=?,category=?,price=?,quantity=?,low_limit=? WHERE id=?",
(name, cat, price, qty, low, pid))
```

```
    con.commit()
    con.close()


def delete_product(pid):
    con = sqlite3.connect("store.db")
    cur = con.cursor()
    cur.execute("DELETE FROM products WHERE id=?", (pid,))
    con.commit()
    con.close()


def get_low_stock():
    con = sqlite3.connect("store.db")
    cur = con.cursor()
    cur.execute("SELECT * FROM products WHERE quantity <= low_limit")
    rows = cur.fetchall()
    con.close()
    return rows


def get_suppliers():
    con = sqlite3.connect("store.db")
    cur = con.cursor()
    cur.execute("SELECT * FROM suppliers")
    rows = cur.fetchall()
    con.close()
    return rows


def add_supplier(name, contact, email):
    con = sqlite3.connect("store.db")
```

```
    cur = con.cursor()
    cur.execute("INSERT INTO suppliers VALUES (NULL,?,?,?)", (name, contact, email))
    con.commit()
    con.close()


def get_movements():
    con = sqlite3.connect("store.db")
    cur = con.cursor()
    cur.execute("SELECT * FROM movements")
    rows = cur.fetchall()
    con.close()
    return rows


def add_movement(product, mtype, qty, date):
    con = sqlite3.connect("store.db")
    cur = con.cursor()
    cur.execute("INSERT INTO movements VALUES (NULL,?,?,?,?)", (product, mtype, qty, date))
    if mtype == "IN":
        cur.execute("UPDATE products SET quantity=quantity+? WHERE name=?", (qty, product))
    else:
        cur.execute("UPDATE products SET quantity=quantity-? WHERE name=?", (qty, product))
    con.commit()
    con.close()
```

login.py

Authentication screen. A fixed-size window with the CoreStock logo, username/password fields, and a login button. Verifies credentials against the users table and, on success, opens the main Dashboard while closing the login window.

```
from PyQt5.QtWidgets import QWidget, QVBoxLayout, QLabel, QLineEdit, QPushButton, QMessageBox,
QFrame

from PyQt5.QtCore import Qt

from PyQt5.QtGui import QFont

import database



class LoginWindow(QWidget):

    def __init__(self):

        super().__init__()

        self.setWindowTitle("CoreStock")

        self.setFixedSize(380, 450)

        self.setStyleSheet("background: white;")



        layout = QVBoxLayout()

        layout.setContentsMargins(45, 35, 45, 35)

        layout.setSpacing(10)



        logo = QLabel("C")

        logo.setFixedSize(50, 50)

        logo.setAlignment(Qt.AlignCenter)

        logo.setFont(QFont("Segoe UI", 20, QFont.Bold))

        logo.setStyleSheet("background:#6366f1; color:white; border-radius:12px;")



        title = QLabel("CoreStock")

        title.setAlignment(Qt.AlignCenter)

        title.setFont(QFont("Segoe UI", 20, QFont.Bold))



        sub = QLabel("PC Store Management System")
```

```
        sub.setAlignment(Qt.AlignCenter)
        sub.setStyleSheet("color:#a1a1aa; font-size:12px;")

        line = QFrame()
        line.setFrameShape(QFrame.HLine)
        line.setStyleSheet("background:#e8e8ed;")
        line.setFixedHeight(1)

        self.username = QLineEdit()
        self.username.setPlaceholderText("Username")
        self.username.setFixedHeight(40)

        self.password = QLineEdit()
        self.password.setPlaceholderText("Password")
        self.password.setEchoMode(QLineEdit.Password)
        self.password.setFixedHeight(40)

        btn = QPushButton("Login")
        btn.setFixedHeight(42)
        btn.setStyleSheet("background:#6366f1; color:white; border-radius:7px; font-size:13px; font-
weight:bold; border:none;")
        btn.clicked.connect(self.do_login)

        hint = QLabel("admin / admin123")
        hint.setAlignment(Qt.AlignCenter)
        hint.setStyleSheet("color:#ccc; font-size:11px;")

        layout.addWidget(logo, alignment=Qt.AlignCenter)
```

```
        layout.addWidget(title)
        layout.addWidget(sub)
        layout.addWidget(line)
        layout.addSpacing(5)
        layout.addWidget(self.username)
        layout.addWidget(self.password)
        layout.addSpacing(8)
        layout.addWidget(btn)
        layout.addWidget(hint)
        self.setLayout(layout)


    def do_login(self):
        u = self.username.text().strip()
        p = self.password.text().strip()
        user = database.check_login(u, p)
        if user:
            from dashboard import Dashboard
            self.dash = Dashboard(user)
            self.dash.show()
            self.close()
        else:
            QMessageBox.warning(self, "Error", "Wrong username or password!")
```

dashboard.py

Main application shell. Builds the dark header bar, the four live summary cards (Products, Total Units, Low Stock, Suppliers), and a tabbed interface hosting the Products, Stock In/Out, Suppliers, and Reports pages. Also handles logout.

```
from PyQt5.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton, QTabWidget,
QFrame
```

```
from PyQt5.QtGui import QFont

from PyQt5.QtCore import Qt

import database



class Dashboard(QWidget):

    def __init__(self, user):

        super().__init__()

        self.user = user

        self.setWindowTitle("CoreStock")

        self.setMinimumSize(900, 560)



        layout = QVBoxLayout()

        layout.setContentsMargins(0, 0, 0, 0)

        layout.setSpacing(0)



        # header

        header = QFrame()

        header.setFixedHeight(54)

        header.setStyleSheet("background:#1c1c1e;")

        h = QHBoxLayout(header)

        h.setContentsMargins(16, 0, 16, 0)



        logo = QLabel("C")

        logo.setFixedSize(30, 30)

        logo.setAlignment(Qt.AlignCenter)

        logo.setFont(QFont("Segoe UI", 13, QFont.Bold))

        logo.setStyleSheet("background:#6366f1; color:white; border-radius:8px;")
```

```
        name = QLabel("CoreStock")
        name.setFont(QFont("Segoe UI", 13, QFont.Bold))
        name.setStyleSheet("color:white;")

        sub = QLabel("PC Store Management")
        sub.setStyleSheet("color:#555; font-size:11px;")

        who = QLabel(user[1] + "  |  " + user[3])
        who.setStyleSheet("color:#888; font-size:11px;")

        out = QPushButton("Logout")
        out.setFixedSize(68, 26)
        out.setStyleSheet("background:#6366f1; color:white; border-radius:6px; font-size:11px;
border:none;")
        out.clicked.connect(self.logout)

        h.addWidget(logo)
        h.addSpacing(8)
        h.addWidget(name)
        h.addSpacing(6)
        h.addWidget(sub)
        h.addStretch()
        h.addWidget(who)
        h.addSpacing(10)
        h.addWidget(out)

        # stats
        stats = QFrame()
```

```
        stats.setFixedHeight(84)
        stats.setStyleSheet("background:#f8f8fa; border-bottom:1px solid #e8e8ed;")
        s = QHBoxLayout(stats)
        s.setContentsMargins(16, 10, 16, 10)
        s.setSpacing(10)

        prods = database.get_products()
        low   = database.get_low_stock()
        sups  = database.get_suppliers()
        units = sum(p[4] for p in prods)

        for title, val, color in [
            ("Products",    len(prods), "#6366f1"),
            ("Total Units", units,      "#1c1c1e"),
            ("Low Stock",   len(low),   "#ef4444"),
            ("Suppliers",   len(sups),  "#1c1c1e"),
        ]:
            card = QFrame()
            card.setStyleSheet("background:white; border-radius:9px; border:1px solid #e8e8ed;")
            cl = QVBoxLayout(card)
            cl.setContentsMargins(12, 6, 12, 6)
            n = QLabel(str(val))
            n.setFont(QFont("Segoe UI", 18, QFont.Bold))
            n.setStyleSheet(f"color:{color};")
            l = QLabel(title)
            l.setStyleSheet("color:#aaa; font-size:10px;")
            cl.addWidget(n)
            cl.addWidget(l)
            s.addWidget(card)
```

```
        # tabs
        tabs = QTabWidget()
        from products  import ProductsPage
        from stock     import StockPage
        from suppliers import SuppliersPage
        from reports   import ReportsPage
        tabs.addTab(ProductsPage(),  "Products")
        tabs.addTab(StockPage(),     "Stock In/Out")
        tabs.addTab(SuppliersPage(), "Suppliers")
        tabs.addTab(ReportsPage(),   "Reports")

        layout.addWidget(header)
        layout.addWidget(stats)
        layout.addWidget(tabs)
        self.setLayout(layout)

    def logout(self):
        from login import LoginWindow
        self.w = LoginWindow()
        self.w.show()
        self.close()
```

products.py

Inventory (Products) module. Displays all products in a searchable table, with Add/Edit/Delete actions through a shared PForm dialog. Includes a live low-stock alert banner refreshed via refresh_alert() every time the table reloads.

```
from PyQt5.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton,
```

```
                              QLineEdit, QTableWidget, QTableWidgetItem, QMessageBox,
                              QDialog, QFormLayout, QComboBox, QSpinBox, QDoubleSpinBox)
from PyQt5.QtCore import Qt
import database


class ProductsPage(QWidget):
    def __init__(self):
        super().__init__()
        layout = QVBoxLayout()
        layout.setContentsMargins(14, 14, 14, 14)


        top = QHBoxLayout()
        top.addWidget(QLabel("Products"))
        top.addStretch()
        a = QPushButton("+ Add Product")
        a.clicked.connect(self.add)
        top.addWidget(a)


        self.alert = QLabel()
        self.search = QLineEdit()
        self.search.setPlaceholderText("Search...")
        self.search.textChanged.connect(self.do_search)


        self.table = QTableWidget()
        self.table.setColumnCount(6)
        self.table.setHorizontalHeaderLabels(["ID","Name","Category","Price","Qty","Low Limit"])
        self.table.setEditTriggers(QTableWidget.NoEditTriggers)
        self.table.setSelectionBehavior(QTableWidget.SelectRows)
```

```
        self.table.verticalHeader().setVisible(False)

        self.table.horizontalHeader().setStretchLastSection(True)



        bot = QHBoxLayout()

        bot.addStretch()

        e = QPushButton("Edit")

        e.clicked.connect(self.edit)

        d = QPushButton("Delete")

        d.setStyleSheet("background:#ef4444; color:white; border-radius:7px; padding:6px 14px; font-
weight:bold; border:none;")

        d.clicked.connect(self.delete)

        bot.addWidget(e)

        bot.addWidget(d)



        layout.addLayout(top)

        layout.addWidget(self.alert)

        layout.addWidget(self.search)

        layout.addWidget(self.table)

        layout.addLayout(bot)

        self.setLayout(layout)

        self.load()



    def refresh_alert(self):

        low = database.get_low_stock()

        if low:

            self.alert.setText("⚠   Low Stock: " + ", ".join(p[1] for p in low))

            self.alert.setStyleSheet("background:#fef2f2; color:#dc2626; border:1px solid #fca5a5;
border-radius:6px; padding:7px;")

        else:
```

```
            self.alert.setText("✓  All stock levels are fine")
            self.alert.setStyleSheet("background:#f0fdf4; color:#16a34a; border:1px solid #86efac;
border-radius:6px; padding:7px;")


    def load(self):
        self.refresh_alert()
        self.fill(database.get_products())


    def fill(self, data):
        self.table.setRowCount(0)
        for row in data:
            r = self.table.rowCount()
            self.table.insertRow(r)
            for c, v in enumerate(row):
                i = QTableWidgetItem(str(v or ""))
                i.setTextAlignment(Qt.AlignCenter)
                self.table.setItem(r, c, i)


    def do_search(self):
        w = self.search.text().lower()
        self.fill([p for p in database.get_products() if w in p[1].lower() or w in (p[2] or
"").lower()])


    def add(self):
        d = PForm(self)
        if d.exec_(): self.load()


    def edit(self):
        row = self.table.currentRow()
```

```
        if row == -1:
            QMessageBox.warning(self, "!", "Select a product first!"); return
        data = [self.table.item(row, i).text() for i in range(6)]
        d = PForm(self, data)
        if d.exec_(): self.load()

    def delete(self):
        row = self.table.currentRow()
        if row == -1:
            QMessageBox.warning(self, "!", "Select a product first!"); return
        pid  = int(self.table.item(row, 0).text())
        name = self.table.item(row, 1).text()
        if QMessageBox.question(self, "Delete?", f"Delete '{name}'?") == QMessageBox.Yes:
            database.delete_product(pid)
            self.load()

class PForm(QDialog):
    def __init__(self, parent, data=None):
        super().__init__(parent)
        self.data = data
        self.setWindowTitle("Add Product" if not data else "Edit Product")
        self.setFixedSize(320, 290)
        self.setStyleSheet("background:white;")
        layout = QFormLayout()
        layout.setContentsMargins(20, 20, 20, 20)
        layout.setSpacing(10)
        self.name  = QLineEdit()
        self.cat   = QComboBox()
```

```
self.cat.addItems(["GPU","CPU","RAM","Monitor","Storage","Motherboard","Keyboard","Mouse","Other"])
        self.price = QDoubleSpinBox()
        self.price.setMaximum(999999)
        self.qty   = QSpinBox()
        self.qty.setMaximum(99999)
        self.low   = QSpinBox()
        self.low.setValue(5)
        if data:
            self.name.setText(data[1])
            self.cat.setCurrentText(data[2])
            self.price.setValue(float(data[3] or 0))
            self.qty.setValue(int(data[4] or 0))
            self.low.setValue(int(data[5] or 5))
        layout.addRow("Name:",     self.name)
        layout.addRow("Category:", self.cat)
        layout.addRow("Price:",    self.price)
        layout.addRow("Quantity:", self.qty)
        layout.addRow("Low Alert:",self.low)
        s = QPushButton("Save")
        s.clicked.connect(self.save)
        layout.addRow(s)
        self.setLayout(layout)


    def save(self):
        if not self.name.text().strip():
            QMessageBox.warning(self, "!", "Name is required!"); return
        if self.data:
            database.update_product(int(self.data[0]), self.name.text(), self.cat.currentText(),
self.price.value(), self.qty.value(), self.low.value())
```

```
        else:
            database.add_product(self.name.text(), self.cat.currentText(), self.price.value(),
self.qty.value(), self.low.value())
        self.accept()
```

stock.py

Stock In/Out module. Lets staff record incoming or outgoing stock movements through a dialog, automatically adjusting product quantities in the database and blocking an OUT movement if it would take stock negative.

```
from PyQt5.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton,
                             QTableWidget, QTableWidgetItem, QDialog, QFormLayout,
                             QComboBox, QSpinBox, QMessageBox)
from PyQt5.QtCore import Qt
from datetime import date
import database


class StockPage(QWidget):
    def __init__(self):
        super().__init__()
        layout = QVBoxLayout()
        layout.setContentsMargins(14, 14, 14, 14)
        top = QHBoxLayout()
        top.addWidget(QLabel("Stock In / Out"))
        top.addStretch()
        b = QPushButton("+ Record Movement")
        b.clicked.connect(self.record)
        top.addWidget(b)
```

```
        self.alert = QLabel()
        self.table = QTableWidget()
        self.table.setColumnCount(5)
        self.table.setHorizontalHeaderLabels(["ID","Product","Type","Qty","Date"])
        self.table.setEditTriggers(QTableWidget.NoEditTriggers)
        self.table.verticalHeader().setVisible(False)
        self.table.horizontalHeader().setStretchLastSection(True)
        layout.addLayout(top)
        layout.addWidget(self.alert)
        layout.addWidget(self.table)
        self.setLayout(layout)
        self.load()

    def refresh_alert(self):
        low = database.get_low_stock()
        if low:
            self.alert.setText("⚠  Low Stock: " + ", ".join(p[1] for p in low))
            self.alert.setStyleSheet("background:#fef2f2; color:#dc2626; border:1px solid #fca5a5;
border-radius:6px; padding:7px;")
        else:
            self.alert.setText("✓  All stock levels are fine")
            self.alert.setStyleSheet("background:#f0fdf4; color:#16a34a; border:1px solid #86efac;
border-radius:6px; padding:7px;")

    def load(self):
        self.refresh_alert()
        data = database.get_movements()
        self.table.setRowCount(0)
        for row in data:
```

```
            r = self.table.rowCount()
            self.table.insertRow(r)
            for c, v in enumerate(row):
                i = QTableWidgetItem(str(v or ""))
                i.setTextAlignment(Qt.AlignCenter)
                self.table.setItem(r, c, i)

    def record(self):
        d = SForm(self)
        if d.exec_(): self.load()

class SForm(QDialog):
    def __init__(self, parent):
        super().__init__(parent)
        self.setWindowTitle("Record Movement")
        self.setFixedSize(300, 200)
        self.setStyleSheet("background:white;")
        layout = QFormLayout()
        layout.setContentsMargins(20, 20, 20, 20)
        layout.setSpacing(10)
        self.product = QComboBox()
        self.products = database.get_products()
        for p in self.products:
            self.product.addItem(f"{p[1]} (qty:{p[4]})", p[1])
        self.mtype = QComboBox()
        self.mtype.addItems(["IN","OUT"])
        self.qty = QSpinBox()
        self.qty.setMinimum(1)
        self.qty.setMaximum(9999)
```

```
        s = QPushButton("Save")
        s.clicked.connect(self.save)
        layout.addRow("Product:", self.product)
        layout.addRow("Type:",    self.mtype)
        layout.addRow("Qty:",     self.qty)
        layout.addRow(s)
        self.setLayout(layout)


    def save(self):
        name = self.product.currentData()
        move = self.mtype.currentText()
        qty  = self.qty.value()
        if move == "OUT":
            p = next((x for x in self.products if x[1] == name), None)
            if p and p[4] < qty:
                QMessageBox.warning(self, "!", f"Only {p[4]} in stock!"); return
        database.add_movement(name, move, qty, str(date.today()))
        self.accept()
```

suppliers.py

Supplier directory module. A simple CRUD screen for maintaining supplier name, contact number, and email address, used for procurement reference.

```
from PyQt5.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton,
                             QTableWidget, QTableWidgetItem, QDialog, QFormLayout,
                             QLineEdit, QMessageBox)
from PyQt5.QtCore import Qt
import database
```

```
class SuppliersPage(QWidget):
    def __init__(self):
        super().__init__()
        layout = QVBoxLayout()
        layout.setContentsMargins(14, 14, 14, 14)
        top = QHBoxLayout()
        top.addWidget(QLabel("Suppliers"))
        top.addStretch()
        b = QPushButton("+ Add Supplier")
        b.clicked.connect(self.add)
        top.addWidget(b)
        self.table = QTableWidget()
        self.table.setColumnCount(4)
        self.table.setHorizontalHeaderLabels(["ID","Name","Contact","Email"])
        self.table.setEditTriggers(QTableWidget.NoEditTriggers)
        self.table.verticalHeader().setVisible(False)
        self.table.horizontalHeader().setStretchLastSection(True)
        layout.addLayout(top)
        layout.addWidget(self.table)
        self.setLayout(layout)
        self.load()

    def load(self):
        data = database.get_suppliers()
        self.table.setRowCount(0)
        for row in data:
            r = self.table.rowCount()
            self.table.insertRow(r)
```

```
            for c, v in enumerate(row):
                i = QTableWidgetItem(str(v or ""))
                i.setTextAlignment(Qt.AlignCenter)
                self.table.setItem(r, c, i)

    def add(self):
        d = SupForm(self)
        if d.exec_(): self.load()

class SupForm(QDialog):
    def __init__(self, parent):
        super().__init__(parent)
        self.setWindowTitle("Add Supplier")
        self.setFixedSize(300, 190)
        self.setStyleSheet("background:white;")
        layout = QFormLayout()
        layout.setContentsMargins(20, 20, 20, 20)
        layout.setSpacing(10)
        self.name    = QLineEdit()
        self.contact = QLineEdit()
        self.email   = QLineEdit()
        s = QPushButton("Save")
        s.clicked.connect(self.save)
        layout.addRow("Name:",    self.name)
        layout.addRow("Contact:", self.contact)
        layout.addRow("Email:",   self.email)
        layout.addRow(s)
        self.setLayout(layout)
```

```
    def save(self):
        if not self.name.text().strip():
            QMessageBox.warning(self, "!", "Name required!"); return
        database.add_supplier(self.name.text(), self.contact.text(), self.email.text())
        self.accept()
```

reports.py

Reporting module. Summarizes total stock IN/OUT and transaction counts, displays the full movement history in a table, and allows the report to be exported as a CSV file.

```
from PyQt5.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton,
                             QTableWidget, QTableWidgetItem, QFileDialog, QMessageBox)
from PyQt5.QtCore import Qt
import database


class ReportsPage(QWidget):
    def __init__(self):
        super().__init__()
        layout = QVBoxLayout()
        layout.setContentsMargins(14, 14, 14, 14)
        top = QHBoxLayout()
        top.addWidget(QLabel("Stock Report"))
        top.addStretch()
        e = QPushButton("Export CSV")
        e.clicked.connect(self.export)
        top.addWidget(e)
        data = database.get_movements()
        tin  = sum(m[3] for m in data if m[2] == "IN")
```

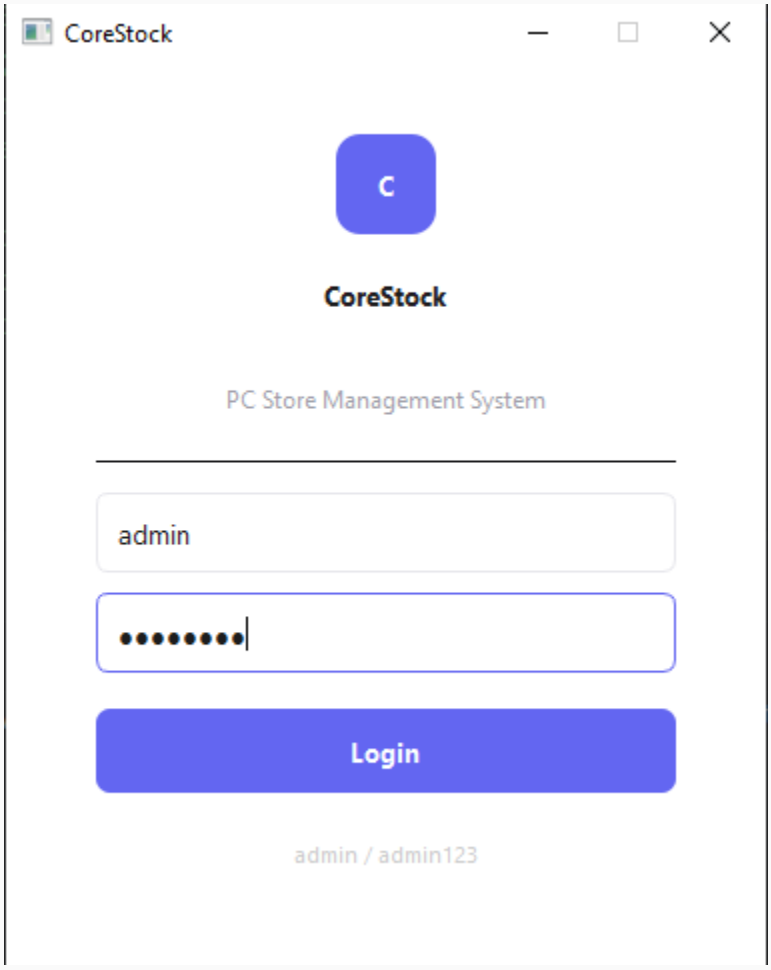
```
        tout = sum(m[3] for m in data if m[2] == "OUT")

        summary = QLabel(f"Total IN: {tin}   |   Total OUT: {tout}   |   Transactions: {len(data)}")

        summary.setStyleSheet("background:#f8f8fa; border:1px solid #e8e8ed; border-radius:6px;
padding:8px; color:#444;")

        self.table = QTableWidget()

        self.table.setColumnCount(5)

        self.table.setHorizontalHeaderLabels(["ID","Product","Type","Qty","Date"])

        self.table.setEditTriggers(QTableWidget.NoEditTriggers)

        self.table.verticalHeader().setVisible(False)

        self.table.horizontalHeader().setStretchLastSection(True)

        layout.addLayout(top)

        layout.addWidget(summary)

        layout.addWidget(self.table)

        self.setLayout(layout)

        self.load()



    def load(self):

        data = database.get_movements()

        self.table.setRowCount(0)

        for row in data:

            r = self.table.rowCount()

            self.table.insertRow(r)

            for c, v in enumerate(row):

                i = QTableWidgetItem(str(v or ""))

                i.setTextAlignment(Qt.AlignCenter)

                self.table.setItem(r, c, i)



    def export(self):

        path, _ = QFileDialog.getSaveFileName(self, "Save", "report.csv", "CSV (*.csv)")
```

```
        if path:
            data = database.get_movements()
            with open(path, "w") as f:
                f.write("ID,Product,Type,Qty,Date\n")
                for row in data:
                    f.write(",".join(str(x or "") for x in row) + "\n")
            QMessageBox.information(self, "Done", "Saved!")
```

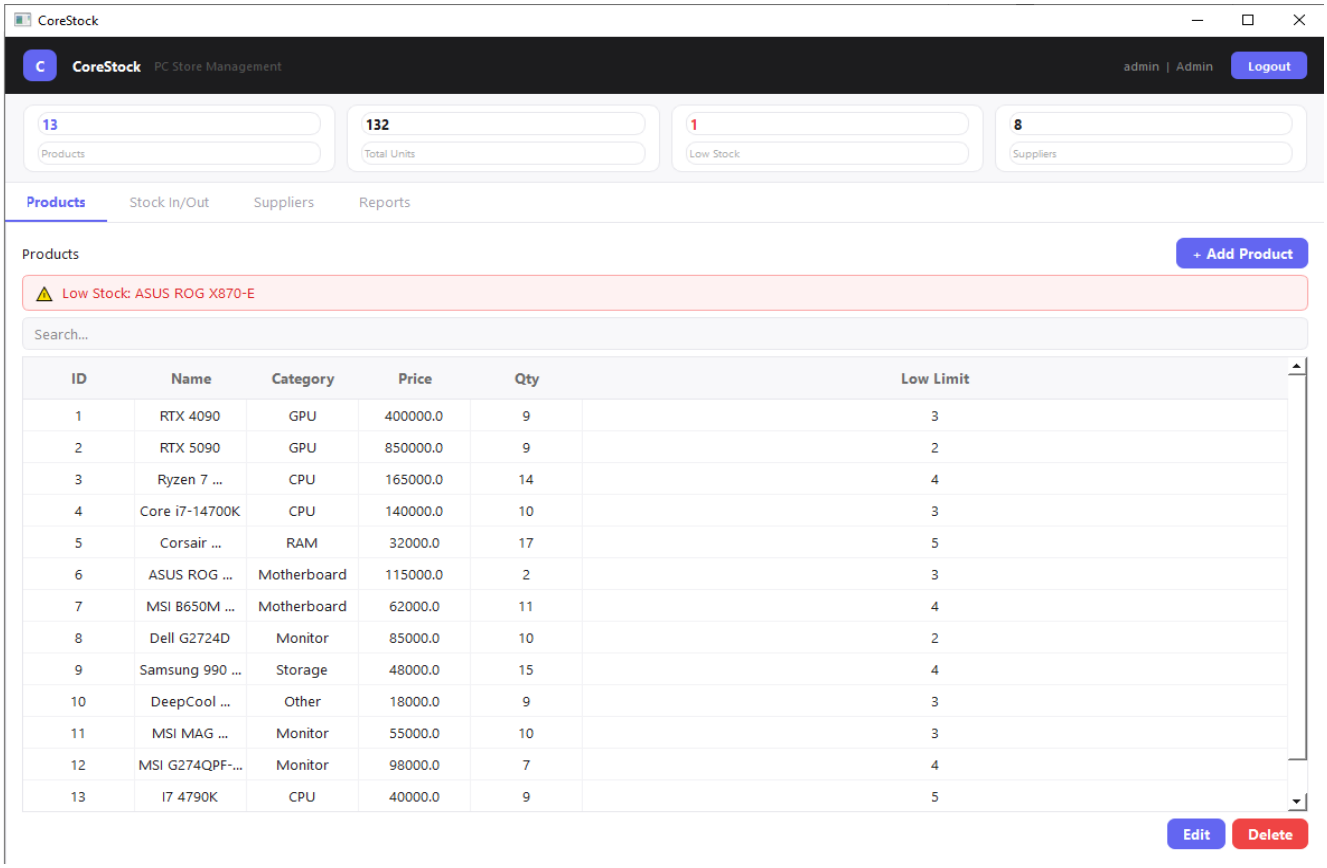
Application Output

Screenshots of the running CoreStock application, one for each screen of the workflow.

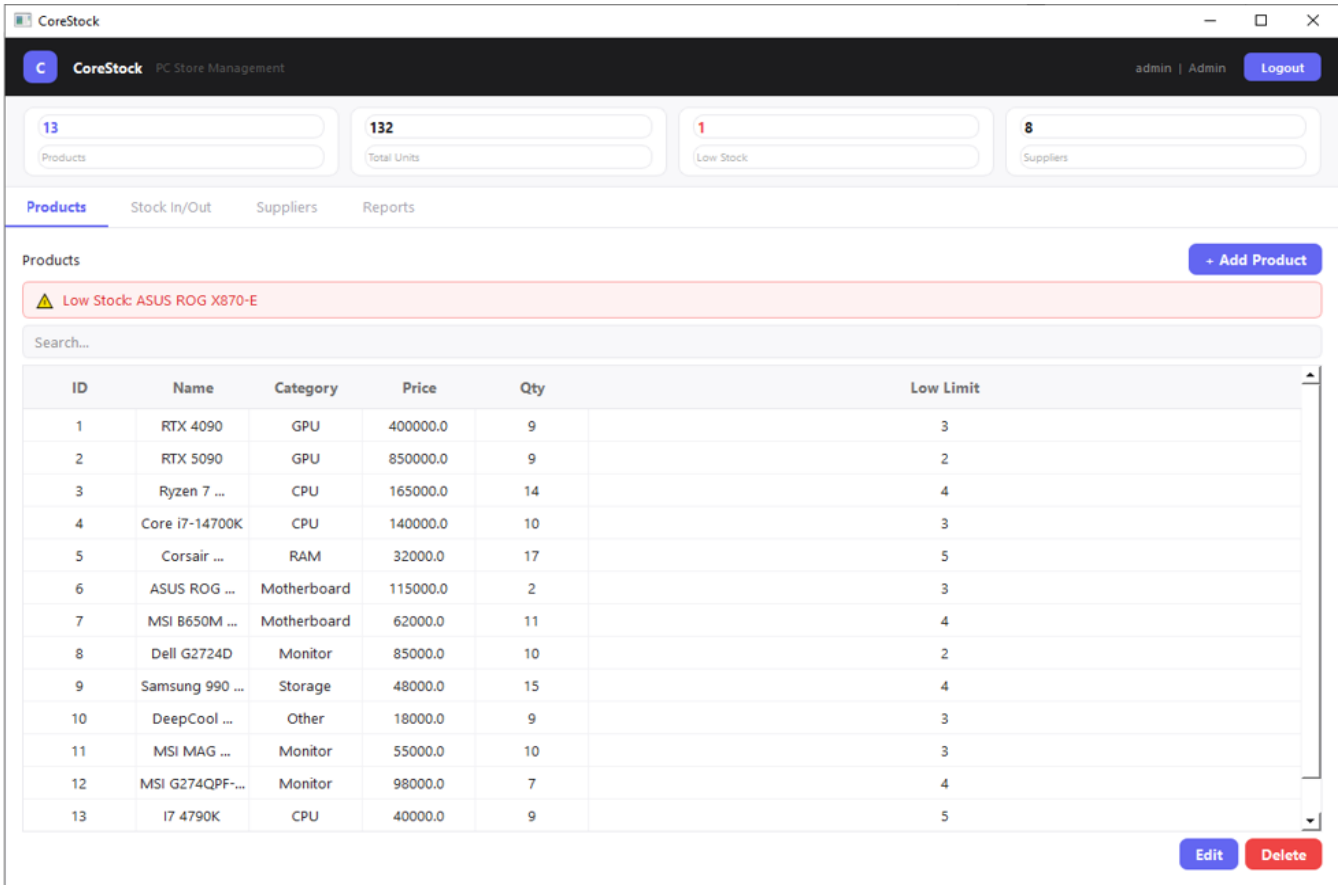
Login Screen



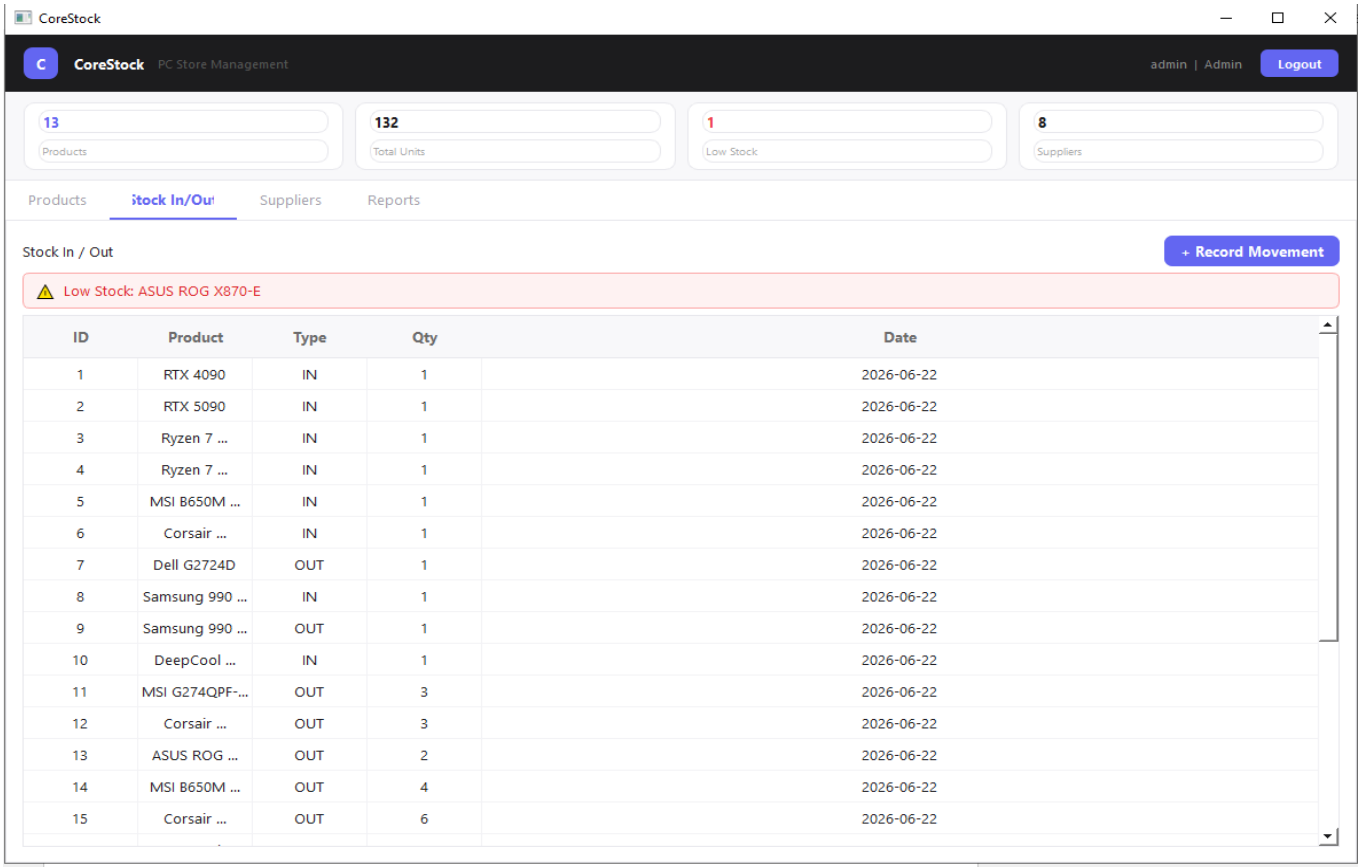
Dashboard — Overview & Summary Cards



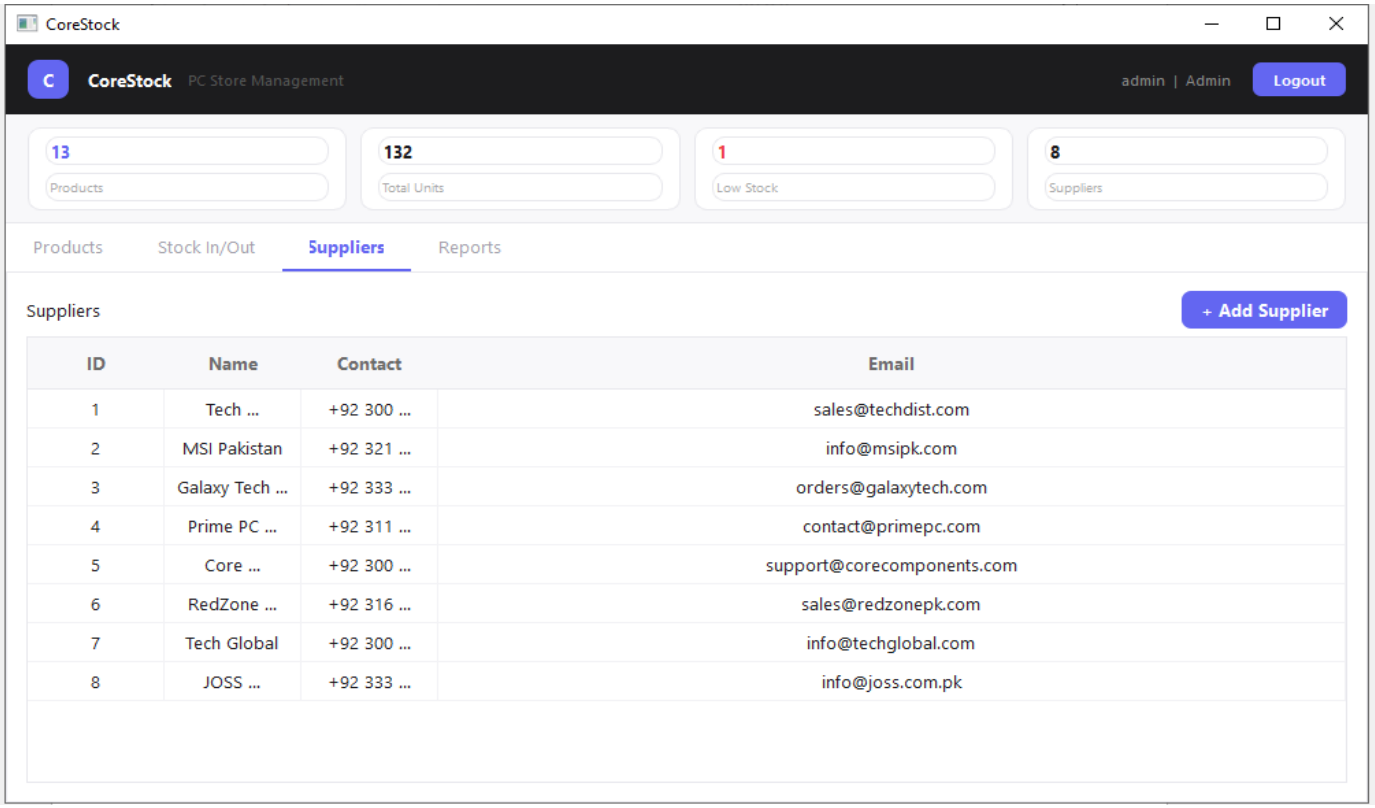
Products Tab — Inventory List



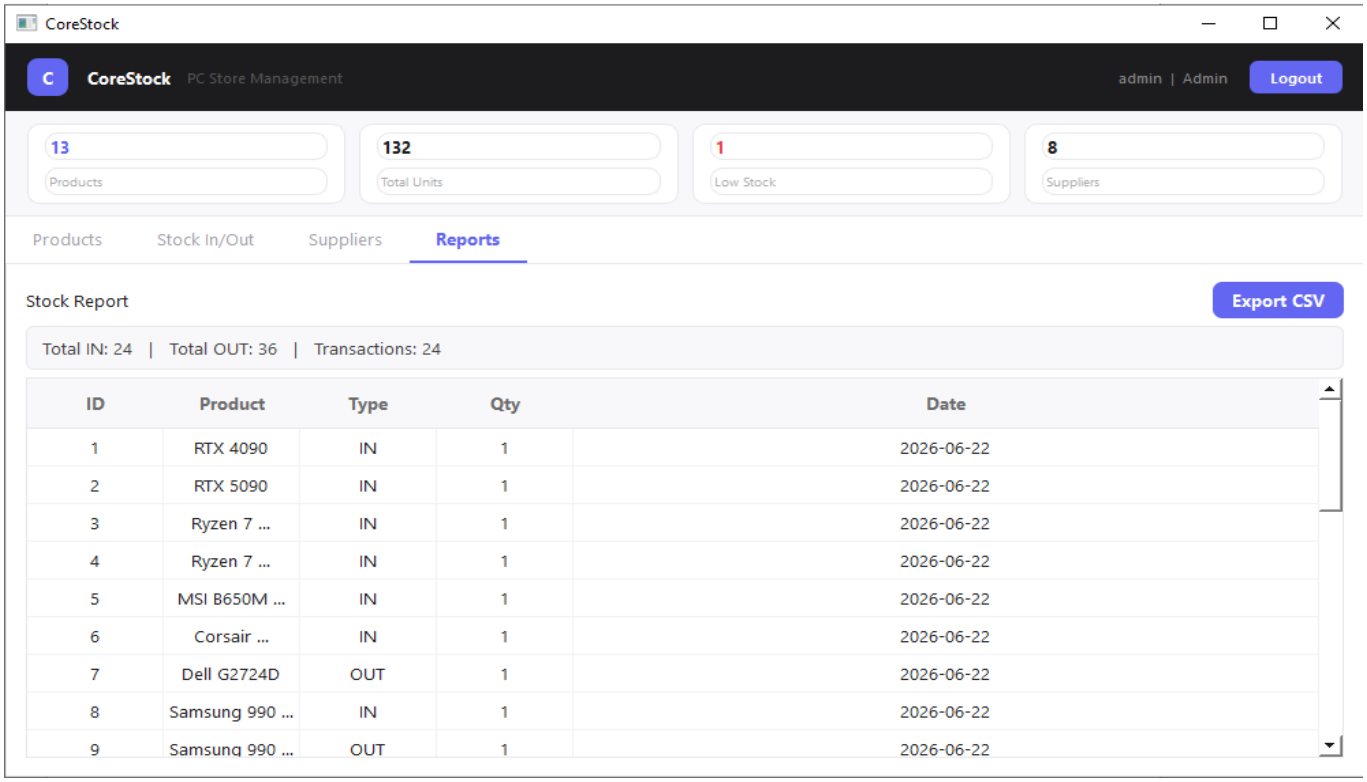
Stock In/Out Tab:



Suppliers Tab



Reports Tab



Conclusions

- Successfully understood the concept of GUI programming with PyQt5
- Learned about Qt Designer and how .ui files relate to Python code
- Understood the core PyQt5 modules: QtCore, QtGui, and QtWidgets
- Mastered the signals and slots mechanism for event handling
- Understood event-driven programming and the main event loop
- Designed and implemented a full relational schema in SQLite for products, suppliers, users, and stock movements
- Built and delivered CoreStock, a complete multi-screen PyQt5 + SQLite inventory management application, as the semester capstone project